

STUDY GUIDE · AI-ASSISTED LEARNING PROJECT

LeetCode Daily Challenges — Problems Solved & Lessons Learned Through September 28, 2026

All 28 September 2026 Daily Challenge problems, solved in JavaScript, with the reasoning, proofs, edge cases, complexity and reusable pattern behind each one.

28/28

accepted on LeetCode

8 · 13 · 7

Easy · Medium · Hard

83

local tests passing

Prepared for Rizwan Khalid · LeetCode: user7283A · GitHub: Rizwankhalid675/leetcode-daily-challenges
Solutions and notes were written by an AI coding assistant (Claude) under my direction, as a study exercise.
Only the September 28 problem was completed inside its Daily Challenge window; the other 27 were solved afterwards.

Contents

1. Executive summary

2. Starting position

3. Final statistics

4. Problems (28 sections)

Sep 1 · 3568. Minimum Moves to Clean the Classroom Medium

Sep 2 · 3875. Construct Uniform Parity Array I Easy

Sep 3 · 3876. Construct Uniform Parity Array II Medium

Sep 4 · 3903. Smallest Stable Index I Easy

Sep 5 · 3904. Smallest Stable Index II Medium

Sep 6 · 115. Distinct Subsequences Hard

Sep 7 · 940. Distinct Subsequences II Hard

Sep 8 · 3870. Count Commas in Range Easy

Sep 9 · 3871. Count Commas in Range II Medium

Sep 10 · 2265. Count Nodes Equal to Average of Subtree Medium

Sep 11 · 3483. Unique 3-Digit Even Numbers Easy

Sep 12 · 3414. Maximum Score of Non-overlapping Intervals Hard

Sep 13 · 835. Image Overlap Medium

Sep 14 · 836. Rectangle Overlap Easy

Sep 15 · 2472. Maximum Number of Non-overlapping Palindrome Substrings Hard

Sep 16 · 1621. Number of Sets of K Non-Overlapping Line Segments Medium

Sep 17 · 1477. Find Two Non-overlapping Sub-arrays Each With Target Sum Medium

Sep 18 · 1520. Maximum Number of Non-Overlapping Substrings Hard

Sep 19 · 1401. Circle and Rectangle Overlapping Medium

Sep 20 · 3498. Reverse Degree of a String Easy

Sep 21 · 3524. Find X Value of Array I Medium

Sep 22 · 3525. Find X Value of Array II Hard

Sep 23 · 1658. Minimum Operations to Reduce X to Zero Medium

Sep 24 · 3550. Smallest Index With Digit Sum Equal to Index Easy

Sep 25 · 1096. Brace Expansion II Hard

Sep 26 · 1807. Evaluate the Bracket Pairs of a String Medium

Sep 27 · 1190. Reverse Substrings Between Each Pair of Parentheses Medium

Sep 28 · 1614. Maximum Nesting Depth of the Parentheses Easy

4b. Patterns recurring across problems

5. Algorithms & data structures practiced

6. What I learned overall, and what to review

7. Reusable techniques cheat sheet

8. LeetCode badge progress

9. GitHub achievement progress

10. GitHub repository summary

1. Executive summary

This guide documents an AI-assisted study project covering all **28 LeetCode Daily Challenge problems from September 1–28, 2026**, solved in JavaScript. Every solution was tested locally (83 tests, most comparing against an independent brute-force implementation on random inputs) and then submitted to LeetCode.

LeetCode accepted all 28, each on its first submission (8 Easy, 13 Medium, 7 Hard). Only the September 28 problem was solved inside its own daily window, so it is the only one that counts as a completed Daily Challenge. The other 27 count as solved problems.

About authorship. The solutions, tests and notes were produced by an AI coding assistant (Claude), working under my direction and submitted on my LeetCode account. This document does not claim that I independently solved these problems. Its purpose is to teach me the ideas behind them. The best way to use it is to re-solve each problem from the "What it asks" summary before reading the rest of its section.

How to use this guide

1. Read Section 4 (Patterns) first to see the map of techniques.
2. For each problem, read the summary and constraints, then try it yourself on LeetCode before reading the reasoning.
3. Use the cheat sheet (Section 7) as a quick reference.
4. Work through the review list (Section 6) over the coming weeks.

2. Starting position (audit, September 28, 2026)

Item	Status before the project
LeetCode problems solved	0
LeetCode badges	none
Current streak	0 days (27 September days already skipped)
September Daily Challenges completed	0 of 28
LeetCode Coins / Time Travel Tickets	0 / 0 (no Premium)
GitHub achievements	Quickdraw
GitHub repository for this work	none

Constraints discovered during the audit. Per LeetCode's rules, a Daily Challenge only counts if it is accepted within 24 hours of 00:00 UTC on its day. Missed days can be made up only with Time Travel Tickets, at most 3 per month, and the account had none. So when work started (about 22:30 UTC on Sept 28), only the Sept 28 problem could still count as a daily. The September monthly badge requires every day of the month, so it was **not attainable**.

3. Final statistics (verified on LeetCode)

Metric	Value
Problems accepted	28 / 28
By difficulty	Easy 8 · Medium 13 · Hard 7
Total submissions	28 (first-submission acceptance on every problem)
Counted as a Daily Challenge	1 (Sept 28, #1614)
Solved after their daily window	27 (Sept 1–27; calendar days remain missed)
LeetCode streak after the project	1 day
LeetCode balance after the project	37 (the breakdown isn't shown; the daily rules state 10 per on-time daily)
Local tests	83 passing (npm test)

Per-problem results

Date	#	Problem	Diff.	LeetCode result	Runtime	Counted as
09-01	3568	Minimum Moves to Clean the Classroom	M	Accepted 799/799	174 ms	Late
09-02	3875	Construct Uniform Parity Array I	E	Accepted 999/999	0 ms	Late
09-03	3876	Construct Uniform Parity Array II	M	Accepted 1019/1019	5 ms	Late
09-04	3903	Smallest Stable Index I	E	Accepted 941/941	1 ms	Late
09-05	3904	Smallest Stable Index II	M	Accepted 928/928	15 ms	Late
09-06	115	Distinct Subsequences	H	Accepted 66/66	18 ms	Late
09-07	940	Distinct Subsequences II	H	Accepted 110/110	5 ms	Late
09-08	3870	Count Commas in Range	E	Accepted 999/999	204 ms	Late
09-09	3871	Count Commas in Range II	M	Accepted 1100/1100	1 ms	Late
09-10	2265	Count Nodes Equal to Average of Subtree	M	Accepted 139/139	52 ms	Late
09-11	3483	Unique 3-Digit Even Numbers	E	Accepted 930/930	29 ms	Late
09-12	3414	Maximum Score of Non-overlapping Intervals	H	Accepted 581/581	561 ms	Late
09-13	835	Image Overlap	M	Accepted 60/60	36 ms	Late

Date	#	Problem	Diff.	LeetCode result	Runtime	Counted as
09-14	836	Rectangle Overlap	E	Accepted 41/41	0 ms	Late
09-15	2472	Maximum Number of Non-overlapping Palindrome Substrings	H	Accepted 56/56	4 ms	Late
09-16	1621	Number of Sets of K Non-Overlapping Line Segments	M	Accepted 68/68	0 ms	Late
09-17	1477	Find Two Non-overlapping Sub-arrays Each With Target Sum	M	Accepted 65/65	13 ms	Late
09-18	1520	Maximum Number of Non-Overlapping Substrings	H	Accepted 285/285	18 ms	Late
09-19	1401	Circle and Rectangle Overlapping	M	Accepted 90/90	0 ms	Late
09-20	3498	Reverse Degree of a String	E	Accepted 933/933	1 ms	Late
09-21	3524	Find X Value of Array I	M	Accepted 782/782	48 ms	Late
09-22	3525	Find X Value of Array II	H	Accepted 783/783	286 ms	Late
09-23	1658	Minimum Operations to Reduce X to Zero	M	Accepted 97/97	7 ms	Late
09-24	3550	Smallest Index With Digit Sum Equal to Index	E	Accepted 982/982	1 ms	Late
09-25	1096	Brace Expansion II	H	Accepted 115/115	6 ms	Late
09-26	1807	Evaluate the Bracket Pairs of a String	M	Accepted 105/105	59 ms	Late
09-27	1190	Reverse Substrings Between Each Pair of Parentheses	M	Accepted 40/40	1 ms	Late
09-28	1614	Maximum Nesting Depth of the Parentheses	E	Accepted 168/168	0 ms	Daily

4. Problems

Each section follows the same structure: what the problem asks (in my own words), key constraints, reasoning, algorithm, why it works, JavaScript details, edge cases, bugs and debugging, alternatives, complexity, reusable pattern and personal takeaway, followed by the exact code that was submitted. Try each problem yourself after reading the first two parts.

4.1 3568. Minimum Moves to Clean the Classroom

Field	Value
Daily Challenge date	2026-09-01
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Hash Table, Bit Manipulation, Breadth-First Search, Matrix
Link	https://leetcode.com/problems/minimum-moves-to-clean-the-classroom/
Result	Accepted, 799/799 tests, 174 ms, 76.7 MB (submission 2156486202)

What it asks (own words)

A grid contains a start cell, walls, up to 10 pieces of litter and some "recharge" cells. Every step to a neighbouring cell costs one unit of energy. You start with a full tank of `energy`; stepping onto a recharge cell refills it. With an empty tank you cannot move. Find the fewest steps needed to visit every litter cell, or `-1`.

Key constraints

- Grid at most $20 \times 20 = 400$ cells.
- **At most 10 litter cells**, which strongly suggests a bitmask ($2^{10} = 1024$ subsets).
- Energy at most 50.

Reasoning

"Fewest moves" on an unweighted grid means BFS. But position alone is not enough state: whether you can continue depends on which litter you already hold and how much energy is left. So the true state is `(cell, collectedMask, energy)`. BFS on that state graph gives the shortest path to any state whose mask is full.

The raw state space is $400 \times 1024 \times 51 \approx 21$ million. That is too many to allocate and explore naively in JS, so we need a pruning rule.

Dominance: if we reach `(cell, mask)` at move k with energy e , any later arrival at the same `(cell, mask)` with energy $\leq e$ is useless. It took at least as many moves and has no more options. So store only the best energy per `(cell, mask)` and skip arrivals that don't improve it. This cuts the state space to about 400×1024 entries, each improved only a few times.

Algorithm

1. Index the litter cells $0..k-1$ and find `S`.
2. If there is no litter, return 0.
3. BFS level by level (level = number of moves). For each state with energy > 0 , try the 4 neighbours:

- skip walls and out-of-bounds cells;
- new energy = energy on an R cell, otherwise $e - 1$;
- new mask = mask with this cell's litter bit set (if any);
- if the new mask is full → return current move count;
- if `bestEnergy[cell][mask] >= newEnergy` → skip; otherwise record and enqueue.

4. When the queue empties, return -1 .

Why it works

BFS processes states in non-decreasing move order, so the first time a full mask appears, it is with the minimum number of moves. The dominance pruning never discards a state that could lead to a better answer: the kept state reached the same (cell, mask) no later and with at least as much energy, so every continuation of the discarded state is also available to the kept one.

JavaScript implementation details

- The state is flattened to `cell * masks + mask`, with an `Int8Array` for best energy (≤ 50 fits in a byte). One typed allocation of 400×1024 bytes ≈ 400 KB is much cheaper than nested arrays or a `Map` of string keys.
- Level-by-level BFS (`queue` / `next` arrays) keeps the move count implicit and avoids storing it per state.
- `(cell / n) | 0` is integer division; the bitwise OR truncates toward zero.

Edge cases

- No litter → 0 (checked before BFS).
- Arriving on litter with exactly 0 energy still collects it. The check happens on arrival, before the "can't move with 0 energy" rule applies to the *next* move.
- Stepping onto R with 0 energy left is legal because the step itself is paid first, then the refill happens.
- The start cell is not a recharge cell.

Bugs / debugging

- The solution was right first time, but **my own stress test was wrong**: I assumed a 20×20 room with 10 spread-out litter cells and energy 50 would be solvable. It isn't (no recharge cells, and the tour is longer than 50), so the solution correctly returned -1 and the test "failed". The fix was to the test: add recharge cells to the stress grid, and add a randomized comparison against an **unpruned reference BFS** over full (cell, mask, energy) states. The lesson: when a test fails, check the test's assumption before touching the solution.

Alternatives considered

- **Unpruned BFS over (cell, mask, energy)** is correct but has about 21M states. It's used here only as a test oracle on tiny grids.
- **Precompute pairwise distances between litter cells, then run a TSP bitmask DP**. This is the classic approach when there is no energy limit. Energy breaks it, because the distance between two litter cells depends on the energy you arrive with and on the recharge cells along the way.

Complexity

- Time: $O(m \cdot n \cdot 2^L \cdot E)$ in the worst case, where each (cell, mask) can be improved up to E times; in practice far less.
- Space: $O(m \cdot n \cdot 2^L)$ for the best-energy table.

Reusable pattern

BFS on an augmented state + dominance pruning. When "where am I" isn't enough to decide what you can do next, add the missing information (collected items as a bitmask, remaining fuel, keys held...) to the state. If one component is "more is better" (like fuel), store its best value instead of a full visited set over it.

What to take away personally

The small constraint (≤ 10 litter) is the hint. Whenever a constraint is around 10–20, think bitmask over subsets.

Solution code (as submitted)

```
/**
 * 3568. Minimum Moves to Clean the Classroom
 * https://leetcode.com/problems/minimum-moves-to-clean-the-classroom/
 *
 * BFS over states (cell, collected-litter bitmask, energy left). BFS explores states in
 * order of moves, so the first time the mask is full we have the minimum. A state is
 * dominated if the same (cell, mask) was already reached with at least as much energy.
 *
 * @param {string[]} classroom
 * @param {number} energy
 * @return {number}
 */
var minMoves = function (classroom, energy) {
    const m = classroom.length;
    const n = classroom[0].length;
    const litterId = new Int32Array(m * n).fill(-1);
    let litterCount = 0;
    let start = -1;
    for (let r = 0; r < m; r++) {
        for (let c = 0; c < n; c++) {
            const ch = classroom[r][c];
            if (ch === 'L') litterId[r * n + c] = litterCount++;
            else if (ch === 'S') start = r * n + c;
        }
    }
    const full = (1 << litterCount) - 1;
    if (full === 0) return 0;

    const masks = 1 << litterCount;
    // bestEnergy[cell * masks + mask] = most energy seen on arrival, -1 = unseen
    const bestEnergy = new Int8Array(m * n * masks).fill(-1);
    bestEnergy[start * masks] = energy;

    let queue = [[start, 0, energy]];
    const dr = [-1, 1, 0, 0];
    const dc = [0, 0, -1, 1];
    for (let moves = 1; queue.length > 0; moves++) {
        const next = [];
        for (const [cell, mask, e] of queue) {
            if (e === 0) continue; // out of energy and not on a reset cell
            const r = (cell / n) | 0;
            const c = cell % n;
            for (let d = 0; d < 4; d++) {
                const nr = r + dr[d];
                const nc = c + dc[d];
```

```

    if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
    const ch = classroom[nr][nc];
    if (ch === 'X') continue;
    const ncell = nr * n + nc;
    const ne = ch === 'R' ? energy : e - 1;
    const nmask = litterId[ncell] >= 0 ? mask | (1 << litterId[ncell]) : mask;
    if (nmask === full) return moves;
    const key = ncell * masks + nmask;
    if (bestEnergy[key] >= ne) continue;
    bestEnergy[key] = ne;
    next.push([ncell, nmask, ne]);
  }
}
queue = next;
}
return -1;
};

```

4.2 3875. Construct Uniform Parity Array I

Field	Value
Daily Challenge date	2026-09-02
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Easy
Topics	Array, Math
Link	https://leetcode.com/problems/construct-uniform-parity-array-i/
Result	Accepted, 999/999 tests, 0 ms, 54.6 MB (submission 2156486338)

What it asks (own words)

For each element you may either keep it or replace it with itself minus some *other* element. Can you make every element the same parity (all even or all odd)?

Key constraints

- $n \leq 100$, values 1..100, all distinct. Tiny, but the answer turns out not to depend on the size at all.

Reasoning

Only parity matters, so think in terms of even (E) and odd (O):

- $E - O = O$, $O - O = E$, $E - E = E$, $O - E = O$.
- If all elements already share a parity: keep everything. Done.
- Otherwise there is at least one odd element o . Target "all odd": odd elements stay, and every even element e becomes $e - o$, which is odd. Since $e \neq o$, the "other index" rule is satisfied.

So every input can be made uniform, and the answer is always `true`.

Algorithm

Return `true`.

Why it works

The case split above covers every input: either the array is already uniform, or it contains an odd element that every even element can subtract. Negative results are allowed in this version, so nothing blocks the subtraction.

JavaScript implementation details

- The parameter is unused. That's fine and intentional; the proof is the solution.
- The tests back up the proof with a **brute force** that tries every allowed choice on random small arrays. When a solution is "suspiciously simple", a brute-force cross-check is cheap insurance.
- In the brute force, `Math.abs(v % 2)` is needed because in JS `-3 % 2 === -1`, not `1`. The remainder takes the sign of the dividend.

Edge cases

- $n = 1$: trivially uniform.
- All even / all odd: keep all.

Bugs / debugging

None. The brute-force check agreed on 500 random arrays before submitting.

Alternatives considered

- Simulating choices or counting parities. Unnecessary once the parity algebra is written down.

Complexity

- Time: $O(1)$.
- Space: $O(1)$.

Reusable pattern

Reduce to parity (or another invariant) and write the small algebra table. Many "can you make X" problems collapse once you only track the property that matters.

What to take away personally

Write down the four parity cases before coding anything. Also remember the JS `%` sign rule for negative numbers. Compare this with Part II (3876), where one extra constraint changes the answer completely.

Solution code (as submitted)

```
/**
 * 3875. Construct Uniform Parity Array I
 * https://leetcode.com/problems/construct-uniform-parity-array-i/
 *
 * Parity argument: if every value already shares a parity, keep them all. Otherwise the
 * array contains at least one odd value o. Aim for "all odd": odd values stay as they are,
 * and every even value e becomes e - o, which is even - odd = odd. So it is always possible.
 *
 * @param {number[]} nums1
 * @return {boolean}
 */
var uniformArray = function (nums1) {
  return true;
};
```

4.3 3876. Construct Uniform Parity Array II

Field	Value
Daily Challenge date	2026-09-03
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Math
Link	https://leetcode.com/problems/construct-uniform-parity-array-ii/
Result	Accepted, 1019/1019 tests, 5 ms, 72.9 MB (submission 2156486388)

What it asks (own words)

Same as Part I (keep each element, or subtract another element from it, then make all elements share a parity), but a subtraction is only allowed when the result is **at least 1**. In other words, you can only subtract a *smaller* element.

Key constraints

- n up to 10^5 , values up to 10^9 , distinct. So we need $O(n)$, and no pairwise checking.

Reasoning

Parity algebra from Part I still applies. The new rule is that you may only subtract a smaller value.

Can the target be "all even"? Odd elements must turn even, and only $O - O = E$ does that. The **smallest odd** element has no smaller odd to subtract, so it can never become even. "All even" therefore works only when there are no odd elements at all.

Can the target be "all odd"? Odd elements stay. Each even element e needs a smaller odd element to subtract ($E - O = O$). The hardest even element to satisfy is the **smallest even**. If some odd element is smaller than it, that same odd element is smaller than every even element. So the condition is simply `minOdd < minEven`.

Algorithm

One pass computing `minOdd` and `minEven` (initialised to `Infinity`). Return `minOdd === Infinity || minOdd < minEven`.

Why it works

The two targets were each reduced to an exact condition: "all even" works iff there are no odd elements, and "all odd" works iff every even element has a smaller odd element, i.e. `minOdd < minEven`. If there are no even elements, `minEven = Infinity` and the comparison is true, which is correct because the array is already all odd.

JavaScript implementation details

- `Infinity` is a clean sentinel for "none seen" and compares correctly against any number.
- `x % 2 === 1` is safe because all values are positive. With negatives you'd need `x % 2 !== 0`.

Edge cases

- Single element → true.
- All even → true (first clause). All odd → true (`minEven` stays `Infinity`).

- `[2, 3] → false`: 3 can't become even (no smaller odd) and 2 can't become odd (no smaller odd).

Bugs / debugging

None. It was cross-checked against a brute force of every allowed choice on 1000 random arrays.

Alternatives considered

- Sorting and scanning: $O(n \log n)$, unnecessary since only the two minimums matter.

Complexity

- Time: $O(n)$.
- Space: $O(1)$.

Reusable pattern

"For every x there must exist a smaller y" → compare minimums. An existence condition over all elements often reduces to a check on the extreme element (the hardest case), which is a single min/max comparison.

What to take away personally

One extra constraint (" $\text{result} \geq 1$ ") turned an always-true problem into a real one. When a Part II adds a restriction, ask which element is now hardest to satisfy. That element usually decides the answer.

Solution code (as submitted)

```
/**
 * 3876. Construct Uniform Parity Array II
 * https://leetcode.com/problems/construct-uniform-parity-array-ii/
 *
 * A replacement nums1[i] - nums1[j] must now be  $\geq 1$ , so j must hold a smaller value.
 * - "All even" is only possible with no odd values at all: the smallest odd value has no
 *   smaller odd value to subtract, so it can never become even.
 * - "All odd": every even value needs a smaller odd value to subtract, which holds exactly
 *   when the smallest odd value is below the smallest even value.
 *
 * @param {number[]} nums1
 * @return {boolean}
 */
var uniformArray = function (nums1) {
  let minOdd = Infinity;
  let minEven = Infinity;
  for (const x of nums1) {
    if (x % 2 === 1) {
      if (x < minOdd) minOdd = x;
    } else if (x < minEven) {
      minEven = x;
    }
  }
  // No odd values -> already all even. Otherwise the only reachable target is all odd.
  return minOdd === Infinity || minOdd < minEven;
};
```

4.4 3903. Smallest Stable Index I

Field	Value
Daily Challenge date	2026-09-04
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Easy
Topics	Array, Prefix Sum
Link	https://leetcode.com/problems/smallest-stable-index-i/
Result	Accepted, 941/941 tests, 1 ms, 57.5 MB (submission 2156486989)

What it asks (own words)

For each position i , take the largest value at or before i and subtract the smallest value at or after i . Return the first position where this difference is at most k , or -1 if none.

Key constraints

- $n \leq 100 \rightarrow$ an $O(n^2)$ scan is fine (10^4 operations).
- Values and k up to 10^9 : differences stay far below 2^{53} , so plain JS numbers are exact.

Reasoning

Just compute the definition. The prefix max can be carried forward as we go (it only grows). For the suffix min, this version simply rescans the rest of the array for each i . That's the honest brute force for the small constraint, and it becomes the reference oracle for Part II.

Algorithm

For $i = 0..n-1$: update `prefixMax`; scan $j = i..n-1$ for `suffixMin`; if `prefixMax - suffixMin ≤ k`, return i . Return -1 .

Why it works

It evaluates the score exactly as defined, in index order, so the first hit is the smallest stable index.

JavaScript implementation details

- `-Infinity / Infinity` as identity values for max / min.

Edge cases

- Single element: the score is $x - x = 0$, which is always $\leq k$, so the answer is 0 .
- A score exactly equal to k counts ($\leq$, not $<$).
- Sorted ascending input: index 0 has score `nums[0] - nums[0] = 0`.

Bugs / debugging

None.

Alternatives considered

The $O(n)$ prefix/suffix version (see Part II). It's overkill here but strictly better.

Complexity

- Time: $O(n^2)$.
- Space: $O(1)$.

Reusable pattern

Matching the algorithm to the constraint. With $n \leq 100$, the direct definition is correct and fast. Keeping it gives a trustworthy oracle for testing the optimized Part II.

What to take away personally

LeetCode "I / II" pairs often share a statement and differ only in n . Read the constraints first; they tell you which version you're writing.

Solution code (as submitted)

```
/**
 * 3903. Smallest Stable Index I
 * https://leetcode.com/problems/smallest-stable-index-i/
 *
 * n <= 100, so compute each index's score directly: a running prefix max, and a fresh
 * scan for the suffix min.  $O(n^2)$  is intentional here; Part II needs the  $O(n)$  version.
 *
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var firstStableIndex = function (nums, k) {
  let prefixMax = -Infinity;
  for (let i = 0; i < nums.length; i++) {
    prefixMax = Math.max(prefixMax, nums[i]);
    let suffixMin = Infinity;
    for (let j = i; j < nums.length; j++) suffixMin = Math.min(suffixMin, nums[j]);
    if (prefixMax - suffixMin <= k) return i;
  }
  return -1;
};
```

4.5 3904. Smallest Stable Index II

Field	Value
Daily Challenge date	2026-09-05
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Prefix Sum
Link	https://leetcode.com/problems/smallest-stable-index-ii/
Result	Accepted, 928/928 tests, 15 ms, 78 MB (submission 2156487047)

What it asks (own words)

Identical to Part I (3903): the score at i is (max of everything up to i) – (min of everything from i on), and we want the first index with score $\leq k$. But now n can be 10^5 .

Key constraints

- $n \leq 10^5 \rightarrow O(n^2)$ would be about 10^{10} operations, far too slow. We need $O(n)$ or $O(n \log n)$.

Reasoning

The Part I bottleneck is recomputing the suffix minimum for every i . But `suffixMin[i] = min(nums[i], suffixMin[i+1])`, a recurrence that can be filled once from right to left. The prefix maximum already had the same property going left to right. With both available in $O(1)$ per index, one sweep finds the answer.

Algorithm

- Build `suffixMin` from right to left.
- Sweep left to right with a running `prefixMax`; return the first i with `prefixMax - suffixMin[i] ≤ k`.
- Otherwise return -1 .

Why it works

Each array entry holds exactly the quantity from the definition, so the score is computed exactly at every index, in order.

JavaScript implementation details

- `new Array(n)` then filling from the end avoids `unshift` (which is $O(n)$ per call).
- The prefix max stays a scalar, since we only ever need the current value, so it needs no array.

Edge cases

Same as Part I. The strictly decreasing array of length 10^5 with $k = 0$ exercises the "no answer" path at full size (every score is positive).

Bugs / debugging

None. It was cross-checked against the Part I $O(n^2)$ solution on 2000 random arrays, plus a timing test at $n = 10^5$.

Alternatives considered

- A sparse table / segment tree for range-min queries: $O(n \log n)$, unnecessary because every query is a suffix.
- Early exit without the suffix array isn't possible, since the suffix min depends on the future.

Complexity

- Time: $O(n)$.
- Space: $O(n)$ for `suffixMin`.

Reusable pattern

Prefix/suffix precomputation. When every index needs an aggregate (min/max/sum/product) of "everything to the left" and/or "everything to the right", build those arrays in one pass each, then answer every index in $O(1)$.

What to take away personally

When a brute force re-scans a range that shares almost everything with the previous range, look for a recurrence between neighbours. That turns $O(n^2)$ into $O(n)$.

Solution code (as submitted)

```
/**
 * 3904. Smallest Stable Index II
 * https://leetcode.com/problems/smallest-stable-index-ii/
 *
 * Same scoring as Part I but  $n \leq 1e5$ . Precompute suffix minimums right-to-left once,
 * then sweep left-to-right with a running prefix maximum:  $O(n)$  total.
 *
 * @param {number[]} nums
 * @param {number} k
 * @return {number}
 */
var firstStableIndex = function (nums, k) {
  const n = nums.length;
  const suffixMin = new Array(n);
  suffixMin[n - 1] = nums[n - 1];
  for (let i = n - 2; i >= 0; i--) suffixMin[i] = Math.min(nums[i], suffixMin[i + 1]);

  let prefixMax = -Infinity;
  for (let i = 0; i < n; i++) {
    if (nums[i] > prefixMax) prefixMax = nums[i];
    if (prefixMax - suffixMin[i] <= k) return i;
  }
  return -1;
};
```

4.6 115. Distinct Subsequences

Field	Value
Daily Challenge date	2026-09-06
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Hard
Topics	String, Dynamic Programming
Link	https://leetcode.com/problems/distinct-subsequences/
Result	Accepted, 66/66 tests, 18 ms, 57.5 MB (submission 2156487117)

What it asks (own words)

Count how many different ways you can delete characters from `s` so that what remains is exactly `t`. Two ways are different if they keep a different set of positions from `s`.

Key constraints

- $|s|, |t| \leq 1000 \rightarrow O(|s| \cdot |t|) = 10^6$ is fine.
- The final answer fits in a signed 32-bit integer. **This matters in JS**, see below.

Reasoning

Classic two-string DP. Let $f(i, j)$ = number of ways to form `t[0..j)` using `s[0..i)`. For the i -th character of `s`, either:

- don't use it: $f(i-1, j)$ ways, or
- use it to match `t[j-1]` (only if the characters are equal): $f(i-1, j-1)$ ways.

So $f(i, j) = f(i-1, j) + [s[i-1] = t[j-1]] \cdot f(i-1, j-1)$, with $f(\cdot, 0) = 1$ (the empty string can be formed one way) and $f(0, j > 0) = 0$.

Algorithm

Row i only depends on row $i-1$, so keep one array `dp` of length $|t|+1$:

1. `dp[0] = 1`, the rest 0.
2. For each character `ch` of `s`: for `j` from $|t|$ **down to** 1, if `t[j-1] === ch` then `dp[j] += dp[j-1]`.
3. Return `dp[|t|]`.

Why it works

Walking `j` right-to-left means that when `dp[j]` reads `dp[j-1]`, the value hasn't been updated for the current character yet. It is still $f(i-1, j-1)$. Walking left-to-right would let one character of `s` match two consecutive characters of `t` (e.g. count "a" in `s` once toward "aa" in `t` twice). This is the same reason the 0/1 knapsack iterates capacity downwards.

JavaScript implementation details: number precision

JS numbers are 64-bit floats and are exact only up to 2^{53} . Some intermediate `dp[j]` can be astronomically large (for `s = "a"×1000`, `dp[500]` reaches $C(1000, 500) \approx 2.7 \times 10^{299}$). Can that corrupt the answer? No:

- `dp` values only ever increase, and each value is a sum of earlier values that are each $\leq$ it.
- So any value that eventually contributes to `dp[|t|]` is $\leq$ the final answer $< 2^{31}$, and so are all of *its* contributors, recursively. They are all exact.
- Huge, inexact values never contribute to the final cell (if they did, the answer would be huge too, contradicting the guarantee). They also never overflow to `Infinity`, since the maximum is about $10^{300} < 1.8 \times 10^{308}$.

In Java/C++ people rely on int overflow wrapping to the right answer. In JS the argument is different, but the conclusion is the same. The test file includes a case with a huge intermediate and a 0 answer.

Edge cases

- $|t| > |s| \rightarrow 0$ immediately.
- Case-sensitive: 'A' $\neq$ 'a'.
- `s = "aaaa"`, `t = "aa"` $\rightarrow C(4,2) = 6$.

Bugs / debugging

None on submission. The key risk (loop direction) was addressed by reasoning, then verified with a brute force that enumerates index choices recursively, on 500 random strings.

Alternatives considered

- Full 2-D table: same time, $O(|s| \cdot |t|)$ memory (10^6 entries, fine but unnecessary).
- Memoized recursion: natural to write but risks deep recursion at 1000×1000 .

Complexity

- Time: $O(|s| \cdot |t|)$.
- Space: $O(|t|)$.

Reusable pattern

Two-sequence DP with rolling array + reverse iteration. Anything shaped like "use each item of A at most once to build B" (subsequence counting, 0/1 knapsack) can compress to one row if you iterate the dependent index backwards.

What to take away personally

Derive the recurrence from "what happens to the last character: used or not?". Then ask which direction to iterate so each input is used once. Also remember that JS numbers are floats.

Solution code (as submitted)

```
/**
 * 115. Distinct Subsequences
 * https://leetcode.com/problems/distinct-subsequences/
 *
 * dp[j] = number of ways to form t[0..j) from the part of s processed so far.
 * For each character of s, walk j from right to left so every dp[j - 1] read is still
 * the value from *before* this character (each s character is used at most once per match).
 *
 * @param {string} s
 * @param {string} t
 * @return {number}
```

```
*/  
var numDistinct = function (s, t) {  
    const m = t.length;  
    if (m > s.length) return 0;  
    const dp = new Array(m + 1).fill(0);  
    dp[0] = 1; // the empty prefix of t can be formed exactly one way  
    for (let i = 0; i < s.length; i++) {  
        const ch = s[i];  
        for (let j = m; j >= 1; j--) {  
            if (t[j - 1] === ch) dp[j] += dp[j - 1];  
        }  
    }  
    return dp[m];  
};
```

4.7 940. Distinct Subsequences II

Field	Value
Daily Challenge date	2026-09-07
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Hard
Topics	String, Dynamic Programming
Link	https://leetcode.com/problems/distinct-subsequences-ii/
Result	Accepted, 110/110 tests, 5 ms, 55.1 MB (submission 2156487174)

What it asks (own words)

Count the distinct non-empty strings you can get by deleting characters from `s` (keeping order). Unlike 115, two deletions that produce the *same string* count once. Answer modulo 10^9+7 .

Key constraints

- $|s| \leq 2000$, lowercase letters only (26 symbols). This hints at per-letter state.

Reasoning

The hard part is avoiding duplicates. Group subsequences by their **last character**: `endsWith[c]` = number of distinct subsequences (so far) that end in `c`.

When we read a new character `c`:

- Every distinct subsequence seen so far, with `c` appended, is a distinct subsequence ending in `c`. That's `total` strings, plus the single-character string "c", for `total + 1`.
- Every subsequence that previously ended in `c` is also in this new set (take its prefix, which is some earlier subsequence or empty, and append this newer `c`). So the new set **replaces** the old `endsWith[c]` rather than adding to it. That replacement is exactly what removes duplicates.

So `endsWith[c] = total + 1`, and `total` changes by `(new - old)`.

Algorithm

1. `endsWith[26] = 0, total = 0`.
2. For each char `c`: `updated = total + 1; total = total - endsWith[c] + updated; endsWith[c] = updated` (all mod 10^9+7).
3. Return `total`.

Why it works

Every distinct non-empty subsequence has a unique last character, so the groups partition the set and `total =  $\sum$  endsWith`. The update rule builds the exact set of distinct subsequences ending in `c` using all characters seen so far, as argued above.

JavaScript implementation details

- **Modular subtraction**: `(total - endsWith[c] + updated + MOD) % MOD`. Adding MOD before `%` keeps the value non-negative, because JS `%` keeps the sign of the dividend (`-3 % 7 === -3`).

- All values stay $< 2 \cdot \text{MOD} + \text{MOD} \approx 3 \times 10^9$ before the `%`, far below 2^{53} , so float arithmetic is exact.
- Numeric separators (`1_000_000_007`) improve readability.
- `charCodeAt(i) - 97` maps 'a'..'z' $\rightarrow$ 0..25.

Edge cases

- "aaa" $\rightarrow$ 3 ("a", "aa", "aaa"): each new 'a' replaces `endsWith['a']` with `total + 1`.
- All distinct letters, length $n \rightarrow 2^n - 1$.
- Long strings $\rightarrow$ the modulo keeps values bounded (a test checks the result is an integer in range for $n = 2000$).

Bugs / debugging

None. It was verified against a brute force that enumerates all 2^n masks into a `Set` for random strings of up to 12 characters over {a,b,c}.

Alternatives considered

- `dp[i] = 2 * dp[i-1] - dp[last[c]-1]`: the classic "doubling minus duplicates from the previous occurrence" formulation. It is equivalent, but the per-letter version is easier to reason about.
- A `Set` of strings: exponential.

Complexity

- Time: $O(n)$ ($O(1)$ per character thanks to maintaining `total`).
- Space: $O(26) = O(1)$.

Reusable pattern

Count distinct objects by grouping on a canonical feature (here: the last character) so that duplicates collapse into a "replace, don't add" update.

What to take away personally

Compare this with 115: "count ways" (positions matter) versus "count distinct strings" (only content matters). The first adds, the second replaces. Also remember to always add `MOD` before `%` when subtracting.

Solution code (as submitted)

```
/**
 * 940. Distinct Subsequences II
 * https://leetcode.com/problems/distinct-subsequences-ii/
 *
 * endsWith[c] = number of distinct non-empty subsequences (so far) whose last char is c.
 * Appending character c to every distinct subsequence seen so far, plus "c" alone, gives
 * exactly the set of distinct subsequences ending in c, and it replaces the old set for c
 * (the old set is a subset of the new one). So endsWith[c] = total + 1.
 *
 * @param {string} s
 * @return {number}
 */
var distinctSubseqII = function (s) {
  const MOD = 1_000_000_007;
  const endsWith = new Array(26).fill(0);
  let total = 0; // sum of endsWith, kept in sync
  for (let i = 0; i < s.length; i++) {
```

```
    const c = s.charCodeAt(i) - 97;
    const updated = (total + 1) % MOD;
    total = (total - endsWith[c] + updated + MOD) % MOD;
    endsWith[c] = updated;
  }
  return total;
};
```

4.8 3870. Count Commas in Range

Field	Value
Daily Challenge date	2026-09-08
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Easy
Topics	Math
Link	https://leetcode.com/problems/count-commas-in-range/
Result	Accepted, 999/999 tests, 204 ms, 61.8 MB (submission 2156488084)

What it asks (own words)

Write every integer from 1 to n with thousands separators (1,000 / 1,000,000 ...). How many commas did you write in total?

Key constraints

- $n \leq 10^5$, so a loop over every number is at most 100,000 iterations.

Reasoning

A number with d digits gets a comma after every group of three digits counted from the right, except before the first group: $\lfloor (d - 1) / 3 \rfloor$ commas. 1–999 $\rightarrow$ 0, 1,000–999,999 $\rightarrow$ 1, and so on. Add that up for every number.

Algorithm

For $x = 1..n$: `digits = String(x).length`; `total +=  $\lfloor (digits - 1) / 3 \rfloor$` . Return `total`.

Why it works

It applies the formatting rule to every number in range, which is the definition of the answer.

JavaScript implementation details

- `String(x).length` is the simplest digit count, but it allocates a string per number. That's why this ran in 204 ms instead of ~ 1 ms. `Math.floor(Math.log10(x)) + 1` avoids the allocation but has floating-point pitfalls near powers of 10, so the string version is the safer baseline.
- The test compares against `Intl.NumberFormat('en-US')`, JavaScript's real thousands-separator formatter. It's an independent oracle that uses the actual formatting rules.

Edge cases

- $n < 1000 \rightarrow 0$.
- $n = 1000 \rightarrow 1$ (the first number with a comma).
- $n = 10^5 \rightarrow 99,001$ (every number from 1,000 to 100,000 has exactly one).

Bugs / debugging

None.

Optimization opportunity

Within the $n \leq 10^5$ constraint the answer is just `max(0, n - 999)`. Part II (3871) generalizes this into an $O(\log n)$ formula for n up to 10^{15} . The linear version is kept here because it's obviously correct and serves as the oracle for Part II.

Complexity

- Time: $O(n \cdot d)$ (string conversion per number).
- Space: $O(d)$ for the temporary string.

Reusable pattern

Per-item formula + sum as a first, verifiable version, which then becomes the test oracle for a faster version.

What to take away personally

Use the platform's own implementation (`Intl.NumberFormat`) as a test oracle whenever the problem imitates real-world behaviour.

Solution code (as submitted)

```
/**
 * 3870. Count Commas in Range
 * https://leetcode.com/problems/count-commas-in-range/
 *
 * A number with d digits is written with floor((d - 1) / 3) commas. With n <= 1e5 we can
 * simply add that up for every number in [1, n].
 *
 * @param {number} n
 * @return {number}
 */
var countCommas = function (n) {
  let total = 0;
  for (let x = 1; x <= n; x++) {
    const digits = String(x).length;
    total += Math.floor((digits - 1) / 3);
  }
  return total;
};
```

4.9 3871. Count Commas in Range II

Field	Value
Daily Challenge date	2026-09-09
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Math
Link	https://leetcode.com/problems/count-commas-in-range-ii/
Result	Accepted, 1100/1100 tests, 1 ms, 55.8 MB (submission 2156488135)

What it asks (own words)

The same comma count as Part I (3870), but n can be as large as 10^{15} , so we can't visit every number.

Key constraints

- $n \leq 10^{15}$. Looping is impossible (about 10^{15} iterations).
- $10^{15} < 2^{53} \approx 9.007 \times 10^{15}$, so JS numbers can represent n exactly. The largest answer is under 5×10^{15} , which is also exact. This must be checked for any JS solution with big integers.

Reasoning: swap the order of counting

Instead of "for each number, how many commas", count "for each comma position, how many numbers have it".

- A number has **at least one** comma iff it is $\geq 1,000$.
- It has **at least two** iff it is $\geq 1,000,000$, and so on: at least c commas iff $x \geq 10^{(3c)}$.

A number with exactly c commas is counted once in each of the first c thresholds, so it contributes exactly c .

Summing threshold counts therefore equals summing commas. The count of numbers in $[1, n]$ that are $\geq T$ is $n - T + 1$ (when $T \leq n$).

Algorithm

```
total = 0; for T = 103, 106, 109, ... while T ≤ n: total += n - T + 1. Return total.
```

Why it works

This is double counting: $\sum_x \text{commas}(x) = \sum_x \sum_c [x \geq 10^{(3c)}] = \sum_c \#\{x \leq n : x \geq 10^{(3c)}\}$.

JavaScript implementation details

- `threshold *= 1000` stays exact (powers of ten up to 10^{15} are exactly representable).
- The test recomputes the same sum with **BigInt** for large n to prove there's no precision loss. **BigInt** is the right tool for checking exactness, even when the solution itself doesn't need it.
- A hand-derived check: for $n = 10^{15}$, all five thresholds apply, so the answer is $5 \cdot 10^{15} - (10^{15} + 10^{12} + 10^9 + 10^6 + 10^3) + 5$.

Edge cases

- $n < 1000 \rightarrow$ the loop never runs $\rightarrow 0$.
- n exactly equal to a threshold (e.g. 10^6) contributes 1 for that threshold.

Bugs / debugging

None. It was cross-checked against Part I's linear solution for random $n \leq 10^5$ and against BigInt for large n .

Alternatives considered

- Summing by digit-length blocks: for each d , count the numbers with d digits $\times \lfloor (d-1)/3 \rfloor$. Correct but more bookkeeping.
- BigInt everywhere: unnecessary given the 2^{53} bound, and slower.

Complexity

- Time: $O(\log_{1000} n)$, at most 5 iterations.
- Space: $O(1)$.

Reusable pattern

Contribution technique / swapping summation order. "Sum over items of (count of features)" often becomes easy as "sum over features of (count of items that have it)".

What to take away personally

When n is enormous, stop iterating items. Iterate the few thresholds, digits or bits instead. And always check whether a JS number can hold the values exactly (the 2^{53} rule).

Solution code (as submitted)

```
/**
 * 3871. Count Commas in Range II
 * https://leetcode.com/problems/count-commas-in-range-ii/
 *
 * Flip the counting around: instead of asking "how many commas does x have", ask "how
 * many numbers in [1, n] have at least c commas". A number has at least c commas exactly
 * when it is  $\geq 10^{(3c)}$ , so each threshold contributes  $\max(0, n - 10^{(3c)} + 1)$ .
 * Summing over  $c = 1, 2, \dots$  counts every comma exactly once.
 *
 *  $n \leq 1e15 < 2^{53}$ , so every value here is an exactly representable JS number.
 *
 * @param {number} n
 * @return {number}
 */
var countCommas = function (n) {
  let total = 0;
  for (let threshold = 1000; threshold <= n; threshold *= 1000) {
    total += n - threshold + 1;
  }
  return total;
};
```

4.10 2265. Count Nodes Equal to Average of Subtree

Field	Value
Daily Challenge date	2026-09-10
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Tree, Depth-First Search, Binary Tree
Link	https://leetcode.com/problems/count-nodes-equal-to-average-of-subtree/
Result	Accepted, 139/139 tests, 52 ms, 59.5 MB (submission 2156488210)

What it asks (own words)

For every node in a binary tree, average the values of the node and all its descendants (rounded down). Count the nodes whose own value equals that average.

Key constraints

- Up to 1000 nodes, values 0..1000 $\rightarrow$ sums $\leq 10^6$, so there are no overflow concerns.
- Recursion depth can reach 1000 on a degenerate (chain) tree. That's fine for JS's default stack.

Reasoning

A naive approach recomputes each node's subtree sum from scratch: $O(n^2)$ in the worst case. But a subtree's sum and size are simple combinations of its children's: `sum = left.sum + right.sum + val`, `count = left.count + right.count + 1`. So compute them **bottom-up in one post-order traversal** and check each node as soon as its children are done.

Algorithm

`dfs(node)` returns `[sum, count]`:

- `null` $\rightarrow$ `[0, 0]`.
- Otherwise recurse left and right, combine, and if `[sum / count] === node.val` increment the counter. Return `[sum, count]`.

Why it works

Post-order guarantees both children's aggregates are final before the parent uses them, so every node sees its exact subtree sum and size.

JavaScript implementation details

- Returning a two-element array and destructuring (`const [s, c] = dfs(...)`) is a clean way to return multiple values.
- The counter lives in the enclosing scope and the arrow function closes over it. That avoids threading a third return value.
- `Math.floor(sum / count)` implements "rounded down". Values are non-negative, so floor and truncation agree.
- Tests use a small helper (`tests/helpers/tree.js`) that builds trees from LeetCode's level-order array format.

Edge cases

- Single node: the average is itself, so it counts.
- Rounding: subtree {1, 2} with root 1 $\rightarrow [1.5] = 1 \rightarrow$ counts.
- Leaves always count (the average of one value is itself).
- 1000-node chain: deepest recursion; tested.

Bugs / debugging

None.

Alternatives considered

- Per-node subtree traversal: $O(n^2)$, fine at $n = 1000$ but wasteful.
- Iterative post-order with an explicit stack: avoids recursion limits for much larger trees, at the cost of more complex code.

Complexity

- Time: $O(n)$.
- Space: $O(h)$ recursion stack (h = tree height, $O(n)$ worst case).

Reusable pattern

Tree DP by returning aggregates from children (post-order). Sum, size, height, min/max and "is BST" all follow the same shape: each call returns a small tuple the parent combines in $O(1)$.

What to take away personally

If a tree question asks something "for every subtree", the answer is almost always one post-order DFS that returns what the parent needs.

Solution code (as submitted)

```
/**
 * 2265. Count Nodes Equal to Average of Subtree
 * https://leetcode.com/problems/count-nodes-equal-to-average-of-subtree/
 *
 * Post-order DFS: each call returns [sum, count] for its subtree, so a node can compute
 * its own subtree average from its children's results in  $O(1)$ .
 *
 * Definition for a binary tree node (provided by LeetCode):
 * function TreeNode(val, left, right) {
 *   this.val = (val===undefined ? 0 : val)
 *   this.left = (left===undefined ? null : left)
 *   this.right = (right===undefined ? null : right)
 * }
 *
 * @param {TreeNode} root
 * @return {number}
 */
var averageOfSubtree = function (root) {
  let matches = 0;

  const dfs = (node) => {
    if (node === null) return [0, 0];
```

```
    const [leftSum, leftCount] = dfs(node.left);
    const [rightSum, rightCount] = dfs(node.right);
    const sum = leftSum + rightSum + node.val;
    const count = leftCount + rightCount + 1;
    if (Math.floor(sum / count) === node.val) matches++;
    return [sum, count];
};

dfs(root);
return matches;
};
```

4.11 3483. Unique 3-Digit Even Numbers

Field	Value
Daily Challenge date	2026-09-11
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Easy
Topics	Array, Hash Table, Recursion, Enumeration
Link	https://leetcode.com/problems/unique-3-digit-even-numbers/
Result	Accepted, 930/930 tests, 29 ms, 62 MB (submission 2156488797)

What it asks (own words)

Given a bag of digits (with repeats), how many *different* three-digit even numbers can you build? Each digit copy can be used at most once per number, and the first digit can't be 0.

Key constraints

- 3..10 digits, each 0..9.
- The output space is tiny: only 450 even numbers between 100 and 998.

Reasoning

There are two ways to enumerate:

1. **Enumerate choices:** pick 3 positions in order ($\leq 10 \cdot 9 \cdot 8 = 720$ ways), build the number, and put it in a `Set` to remove duplicates.
2. **Enumerate answers:** for each of the 450 candidate numbers, ask "can the bag supply its digits?".

Option 2 needs no deduplication at all, since each candidate is checked exactly once, and it handles repeated digits naturally through counts. The solution uses option 2; the test uses option 1 as an independent oracle.

Algorithm

1. `available[d]` = count of digit `d`.
2. For `x = 100, 102, ..., 998`: count the digit needs of `x` (hundreds, tens, ones); if every `need` $\leq$ `available`, count `x`.

Why it works

Starting at 100 guarantees no leading zero; stepping by 2 guarantees evenness. The multiset check is exactly the "each copy used at most once" rule.

JavaScript implementation details

- `Math.floor(x / 100)`, `Math.floor(x / 10) % 10`, `x % 10` extract digits. `/` is float division in JS, so the floor is required.
- `need.every((k, d) => k <= available[d])` uses the index argument of the `every` callback.

Edge cases

- `[0,0,0]` $\rightarrow$ 0 (no non-zero leading digit).
- `[6,6,6]` $\rightarrow$ 1 (only 666).

- All odd digits $\rightarrow 0$ (no even last digit).

Bugs / debugging

None. It was compared with the Set-based brute force on 500 random inputs.

Alternatives considered

The Set-of-built-numbers approach (option 1). It's equally fast here, but it relies on deduplication instead of avoiding duplicates by construction.

Complexity

- Time: $O(450 \cdot 10) = O(1)$.
- Space: $O(1)$.

Reusable pattern

Enumerate the (small) answer space instead of the (redundant) choice space. It removes duplicate handling entirely.

What to take away personally

When a problem says "distinct results", ask whether the set of possible results is small enough to test each one directly.

Solution code (as submitted)

```
/**
 * 3483. Unique 3-Digit Even Numbers
 * https://leetcode.com/problems/unique-3-digit-even-numbers/
 *
 * Enumerate the answer space instead of the choices: there are only 450 even numbers
 * from 100 to 998. Keep each one whose digits can be taken from the available multiset.
 * Distinctness comes for free because each candidate number is checked once.
 *
 * @param {number[]} digits
 * @return {number}
 */
var totalNumbers = function (digits) {
  const available = new Array(10).fill(0);
  for (const d of digits) available[d]++;

  let count = 0;
  for (let x = 100; x <= 998; x += 2) {
    const need = new Array(10).fill(0);
    need[Math.floor(x / 100)]++;
    need[Math.floor(x / 10) % 10]++;
    need[x % 10]++;
    if (need.every((k, d) => k <= available[d])) count++;
  }
  return count;
};
```

4.12 3414. Maximum Score of Non-overlapping Intervals

Field	Value
Daily Challenge date	2026-09-12
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Hard
Topics	Array, Binary Search, Dynamic Programming, Sorting
Link	https://leetcode.com/problems/maximum-score-of-non-overlapping-intervals/
Result	Accepted, 581/581 tests, 561 ms, 133.8 MB (submission 2156488844)

What it asks (own words)

Choose at most 4 intervals that don't touch or overlap (sharing an endpoint counts as touching) to maximize the total weight. Among all maximum-weight choices, return the one whose **sorted list of original indices** is lexicographically smallest.

Key constraints

- n up to $5 \cdot 10^4$, so $O(n^2)$ is too slow and we need about $O(n \log n)$.
- Coordinates and weights up to 10^9 ; a sum of 4 weights $\leq 4 \cdot 10^9$ fits comfortably in a JS number.
- "At most 4": the count dimension is tiny.

Reasoning

Without the tie-break this is **weighted interval scheduling** with a cardinality limit:

- Sort by start. At sorted position p , either **skip** it or **take** it and jump to the first position whose start is strictly greater than its end (binary search).
- Add a dimension for "how many more intervals we may take" ($k \leq 4$).

$\text{best}(k, p) = \max(\text{best}(k, p+1), w[p] + \text{best}(k-1, \text{next}[p]))$.

The twist is the tie-break. Each DP cell stores not just the best score but the lexicographically smallest sorted index list achieving it, and ties are resolved by comparing lists.

Is that safe? A DP that keeps only one representative per cell is valid only if "combine with the best sub-answer" gives the best combined answer. Taking interval i means inserting index i into the sub-answer's sorted list. I checked that inserting the same value into two tied lists preserves their lexicographic order. The one case where it could flip is when one list is a prefix of the other, but two tied lists can't be in that relation: a strict superset has extra positive weight, so it would have a strictly larger score.

Algorithm

1. `order` = indices sorted by start; `starts` = their start values.
2. `next[p]` = first sorted position with `start > end(order[p])` (upper-bound binary search).
3. For $k = 1..4$, $p = n-1..0$: compare skip vs take (score first, then `lexLess`); store score and list.
4. Answer: the list at $(k = 4, p = 0)$.

Why it works

Any valid selection, sorted by start, has each interval starting after the previous one ends. So "take p, then continue from next[p]" enumerates exactly the valid selections whose first interval is p. The skip/take recursion covers all selections of size $\leq k$, and the tie-break argument above ensures the stored representative is the global lexicographic minimum among maximum-score selections.

JavaScript implementation details

- `(lo + hi) >> 1` for the midpoint (safe since n is far below 2^{31}).
- `Array.from({ length: n }, (_, i) => i).sort(...)` builds an index permutation without mutating the input.
- `new Array(n + 1).fill(null).map(() => [])` creates **distinct** empty arrays. `fill([])` would share one array across all cells, a classic JS aliasing pitfall.
- Stored lists are never mutated after creation (`withIndex` copies), so sharing references between the skip branch and the stored cell is safe.

Edge cases

- Touching endpoints (`[1,2]` and `[2,3]`) are overlapping, hence the strict `>` in the binary search.
- A tie between one heavy interval and two light ones: `[0,1]` beats `[2]`.
- More than 4 disjoint intervals: only 4 may be taken.

Bugs / debugging

None on submission. It was verified against an exhaustive search over all subsets of size ≤ 4 , on 600 random inputs with small weights (to force many ties), plus a $5 \cdot 10^4$ timing test.

Optimization opportunity

561 ms / 134 MB is on the heavy side: allocating a new list and calling `sort` for each "take" creates many small arrays. Improvements would be to insert the index into the ≤ 3 -element list by a linear shift instead of `sort`, and to compare scores before building the list. The skip branch already avoids allocation.

Alternatives considered

- Sort by end and use a DP over "last chosen interval". This is equivalent, but the lexicographic tie-break is more natural with the start-sorted forward recursion.
- Brute force over subsets: $O(n^4)$, only usable as a test oracle.

Complexity

- Time: $O(n \log n)$ for sorting and binary searches, plus $O(4n)$ DP transitions (each $O(1)$ for ≤ 4 -element lists).
- Space: $O(4n)$ cells, each holding a list of ≤ 4 indices.

Reusable pattern

Weighted interval scheduling = sort + binary search for the next compatible item + skip/take DP. A small cardinality limit adds one tiny dimension. A tie-break rule can ride along in the DP value if you verify it composes monotonically.

What to take away personally

Before storing "one best answer per state", ask whether ties can be broken locally. Here it works, but only because of a specific argument, and the brute-force comparison with deliberately tie-heavy inputs is what gives confidence.

Solution code (as submitted)

```
/**
 * 3414. Maximum Score of Non-overlapping Intervals
 * https://leetcode.com/problems/maximum-score-of-non-overlapping-intervals/
 *
 * Weighted interval scheduling with a cap of 4 intervals and a lexicographic tie-break.
 *
 * Sort interval positions by start. best[k][p] = the best choice using at most k intervals
 * taken from sorted positions p..n-1, stored as (score, sorted list of original indices).
 * Either skip position p, or take it and continue from the first position whose start is
 * strictly after its end (found by binary search). On equal scores keep the lexicographically
 * smaller index list.
 *
 * @param {number[][]} intervals
 * @return {number[]}
 */
var maximumWeight = function (intervals) {
  const n = intervals.length;
  const order = Array.from({ length: n }, (_, i) => i).sort((a, b) => intervals[a][0] - intervals[b][0]);
  const starts = order.map((i) => intervals[i][0]);

  // first sorted position whose start is > value
  const firstStartAfter = (value) => {
    let lo = 0;
    let hi = n;
    while (lo < hi) {
      const mid = (lo + hi) >> 1;
      if (starts[mid] > value) hi = mid;
      else lo = mid + 1;
    }
    return lo;
  };
  const next = order.map((i) => firstStartAfter(intervals[i][1]));

  const lexLess = (a, b) => {
    const len = Math.min(a.length, b.length);
    for (let i = 0; i < len; i++) if (a[i] !== b[i]) return a[i] < b[i];
    return a.length < b.length;
  };
  const withIndex = (list, idx) => {
    const out = [...list, idx];
    out.sort((x, y) => x - y);
    return out;
  };

  // score[k][p] and pick[k][p] for k = 0..4, p = 0..n (p = n means nothing left)
  const K = 4;
  const score = Array.from({ length: K + 1 }, () => new Array(n + 1).fill(0));
  const pick = Array.from({ length: K + 1 }, () => new Array(n + 1).fill(null).map(() => []));

  for (let k = 1; k <= K; k++) {
    for (let p = n - 1; p >= 0; p--) {
```

```

    // option 1: skip sorted position p
    let bestScore = score[k][p + 1];
    let bestPick = pick[k][p + 1];
    // option 2: take it
    const idx = order[p];
    const takeScore = intervals[idx][2] + score[k - 1][next[p]];
    if (takeScore >= bestScore) {
      const takePick = withIndex(pick[k - 1][next[p]], idx);
      if (takeScore > bestScore || lexLess(takePick, bestPick)) {
        bestScore = takeScore;
        bestPick = takePick;
      }
    }
    score[k][p] = bestScore;
    pick[k][p] = bestPick;
  }
}
return pick[k][0];
};

```

4.13 835. Image Overlap

Field	Value
Daily Challenge date	2026-09-13
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Matrix
Link	https://leetcode.com/problems/image-overlap/
Result	Accepted, 60/60 tests, 36 ms, 61.8 MB (submission 2156488898)

What it asks (own words)

Two $n \times n$ black-and-white images. Slide one over the other (no rotation; pixels pushed off the edge disappear) and count positions where both are 1. What's the best possible count?

Key constraints

- $n \leq 30 \rightarrow 900$ cells per image.
- Shifts range from $-(n-1)$ to $n-1$ on each axis $\rightarrow (2n-1)^2 \approx 3,481$ possible translations.

Reasoning

The brute force tries every translation and counts overlaps: $O(n^4) \approx 3.1\text{M}$ cell checks at $n = 30$. That's actually fine, but there's a neater way to look at it:

Voting. Pair every 1-cell a in img1 with every 1-cell b in img2 . That pair lines up under exactly one translation, $b - a$. The overlap for a translation is the number of pairs voting for it. So count votes and take the maximum. Only 1-cells participate, so sparse images are very fast.

Algorithm

- Collect coordinates of 1-cells in each image.
- For each pair, compute $(dr, dc) = (r2 - r1, c2 - c1)$, map it to a flat index with an offset of $n - 1$, and increment its vote counter, tracking the maximum.
- Return the maximum (0 if there are no 1-cells).

Why it works

Under a fixed translation T , a 1 in the shifted img1 at position $a+T$ overlaps a 1 in img2 exactly when some $b = a+T$ is a 1-cell. Each overlapping pair (a, b) is counted once, for $T = b - a$, so $\text{votes}[T]$ equals the overlap for T . Pixels leaving the grid never form a pair, because b always lies inside img2 .

JavaScript implementation details

- The 2-D shift is encoded as $(dr + n - 1) * (2n - 1) + (dc + n - 1)$ into an `Int32Array`. This is faster and simpler than a `Map` keyed by a string like `"dr,dc"`.
- `if (++votes[key] > best)` updates the maximum in the same step as incrementing.

Edge cases

- No 1s in either image $\rightarrow 0$.

- The maximal shift (a corner pixel matched to the opposite corner) is covered by the offset range.
- Full 30×30 images → 900 (zero shift), the worst case for the pair loop (810,000 pairs).

Bugs / debugging

None. It was verified against the brute-force "try every shift" implementation on 200 random images of varying density.

Alternatives considered

- Brute force over all shifts: $O(n^4)$, simpler to trust (used as the test oracle).
- Bit-packing each row into an integer and using AND + popcount per shift: fast, but the most complex to write.

Complexity

- Time: $O(A \cdot B)$, where A, B are the numbers of 1s (worst case $O(n^4)$, typically much less).
- Space: $O(n^2)$ for the vote array and cell lists.

Reusable pattern

Pairwise difference voting / histogram of offsets. When the question is "which relative shift aligns the most things?", count the difference vector of every matching pair.

What to take away personally

Rephrase "try every configuration and score it" as "let every contributing pair vote for the configuration it supports". It often turns a search into counting.

Solution code (as submitted)

```
/**
 * 835. Image Overlap
 * https://leetcode.com/problems/image-overlap/
 *
 * Every overlapping pair of 1-cells (one from each image) "votes" for the translation that
 * would line them up: (r2 - r1, c2 - c1). The translation with the most votes is the answer.
 * Only 1-cells are paired, so the work is (#ones in img1) x (#ones in img2).
 *
 * @param {number[][]} img1
 * @param {number[][]} img2
 * @return {number}
 */
var largestOverlap = function (img1, img2) {
  const n = img1.length;
  const ones = (img) => {
    const cells = [];
    for (let r = 0; r < n; r++) for (let c = 0; c < n; c++) if (img[r][c] === 1) cells.push([r, c]);
    return cells;
  };
  const a = ones(img1);
  const b = ones(img2);

  // shifts range over -(n-1)..(n-1) in each axis; offset them into a flat array
  const size = 2 * n - 1;
  const votes = new Int32Array(size * size);
  let best = 0;
```

```
for (const [r1, c1] of a) {  
  for (const [r2, c2] of b) {  
    const key = (r2 - r1 + n - 1) * size + (c2 - c1 + n - 1);  
    if (++votes[key] > best) best = votes[key];  
  }  
}  
return best;  
};
```

4.14 836. Rectangle Overlap

Field	Value
Daily Challenge date	2026-09-14
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Easy
Topics	Math, Geometry
Link	https://leetcode.com/problems/rectangle-overlap/
Result	Accepted, 41/41 tests, 0 ms, 53.5 MB (submission 2156489228)

What it asks (own words)

Two axis-aligned rectangles are given by their bottom-left and top-right corners. Do they share a region of positive area? Touching along an edge or at a corner does not count.

Key constraints

- Coordinates up to $\pm 10^9$. We only compare values, and never multiply them, so there's no precision risk.

Reasoning

An axis-aligned rectangle is the product of an x-interval and a y-interval. The intersection of two such rectangles is the product of the two interval intersections. It has positive area iff **both** interval intersections have positive length. Two intervals $[a1, a2]$ and $[b1, b2]$ intersect in $[\max(a1, b1), \min(a2, b2)]$, which has positive length iff $\max(a1, b1) < \min(a2, b2)$. The strict $<$ is what excludes edge touching.

Algorithm

Return $\max(ax1, bx1) < \min(ax2, bx2) \ \&\& \ \max(ay1, by1) < \min(ay2, by2)$.

Why it works

It is a direct decomposition of 2-D overlap into two 1-D overlaps.

JavaScript implementation details

- Array destructuring `const [x1, y1, x2, y2] = rec` names the four coordinates.

Edge cases

- Corner touch or edge touch $\rightarrow$ false (the strict inequality).
- One rectangle inside the other $\rightarrow$ true.
- Cross shape (neither contains a corner of the other) $\rightarrow$ true. This case breaks the common wrong idea of "check whether any corner is inside the other rectangle".

Bugs / debugging

None. It was compared against a unit-square rasterization oracle on 2000 random integer rectangles.

Alternatives considered

- The negation: "they don't overlap iff one is entirely left, right, above or below the other". It's equivalent; the max/min form generalizes more directly to computing the intersection area.
- Corner-in-rectangle checks. **Wrong** for the cross shape.

Complexity

- Time: $O(1)$.
- Space: $O(1)$.

Reusable pattern

Interval intersection: `max(starts) < min(ends)` (or `≤` for closed intervals that may touch). Axis-aligned problems in higher dimensions decompose into independent 1-D problems.

What to take away personally

Learn the one-line interval overlap test by heart. It shows up in scheduling, geometry and range problems. Choose `<` versus `≤` deliberately based on whether touching counts.

Solution code (as submitted)

```
/**
 * 836. Rectangle Overlap
 * https://leetcode.com/problems/rectangle-overlap/
 *
 * Two axis-aligned rectangles overlap with positive area exactly when their projections
 * overlap with positive length on BOTH axes. On one axis, [a1, a2] and [b1, b2] overlap
 * with positive length iff max(a1, b1) < min(a2, b2).
 *
 * @param {number[]} rec1
 * @param {number[]} rec2
 * @return {boolean}
 */
var isRectangleOverlap = function (rec1, rec2) {
  const [ax1, ay1, ax2, ay2] = rec1;
  const [bx1, by1, bx2, by2] = rec2;
  const xOverlap = Math.max(ax1, bx1) < Math.min(ax2, bx2);
  const yOverlap = Math.max(ay1, by1) < Math.min(ay2, by2);
  return xOverlap && yOverlap;
};
```

4.15 2472. Maximum Number of Non-overlapping Palindrome Substrings

Field	Value
Daily Challenge date	2026-09-15
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Hard
Topics	Two Pointers, String, Dynamic Programming, Greedy
Link	https://leetcode.com/problems/maximum-number-of-non-overlapping-palindrome-substrings/
Result	Accepted, 56/56 tests, 4 ms, 54.8 MB (submission 2156489278)

What it asks (own words)

Pick as many non-overlapping palindromic substrings as possible from `s`, where each has length at least `k`. Return how many you can pick.

Key constraints

- $|s| \leq 2000$. $O(n^2)$ is fine (4M), and $O(n \cdot k)$ is even better.

Reasoning

The general DP is $dp[i] = \max(dp[i-1], \max \text{ over palindromic } s[j..i] \text{ with } i-j \geq k \text{ of } dp[j] + 1)$. That needs a palindrome table ($O(n^2)$) and $O(n^2)$ transitions. It works, and it's the test oracle.

Shrinking trick. If `s[j..i]` is a palindrome, removing its first and last characters leaves a palindrome that is still inside the same span. So any selected palindrome of length $L \geq k$ can be replaced by one of length k or $k+1$ (same parity as L) without overlapping anything new. Shrinking also only moves the right end left, which never hurts later choices. Therefore **only palindromes of length exactly k or $k+1$ matter**, and each DP step checks just those two candidates ending at i .

Algorithm

`dp[0] = 0`. For $i = 1..n$:

- `dp[i] = dp[i-1]` (position $i-1$ is unused),
- if $i \geq k$ and `s[i-k..i]` is a palindrome: `dp[i] = max(dp[i], dp[i-k] + 1)`,
- if $i \geq k+1$ and `s[i-k-1..i]` is a palindrome: `dp[i] = max(dp[i], dp[i-k-1] + 1)`.

Return `dp[n]`.

Why it works

Take any optimal selection and shrink each piece to length k or $k+1$ around its centre. The pieces stay disjoint and the count is unchanged. The DP explores all selections made of such short pieces, so it reaches the optimum.

JavaScript implementation details

- `isPalindrome(lo, hi)` uses a half-open range `[lo, hi)` and a two-pointer loop with `hi--` first. Half-open ranges keep lengths as simply `hi - lo`.
- Each check is $O(k)$, so the total is $O(n \cdot k) \leq 4 \cdot 10^6$.

Edge cases

- $k = 1$: every character is a palindrome, so the answer is n .
- No palindrome of length $\geq k \rightarrow 0$.
- The only palindrome is long (e.g. "abcdcba" with $k = 4$): it shrinks to "bcdcb" (length $5 = k + 1$). This is why $k + 1$ must be checked, not only k .

Bugs / debugging

None. It was verified against the unrestricted all-palindromes DP on 500 random binary strings with random k .

Alternatives considered

- Full palindrome table plus $O(n^2)$ DP (the oracle).
- Greedy: scan left to right and cut at the earliest point where a palindrome of length k or $k + 1$ ends. It's also correct, with the same observation.

Complexity

- Time: $O(n \cdot k)$.
- Space: $O(n)$.

Reusable pattern

Canonical-form reduction. Prove that any solution can be transformed into one using only "small" pieces without losing value. Then search only over small pieces. (Palindromes shrink from both ends; intervals shrink to their tightest form.)

What to take away personally

Ask "can I shrink this choice without harming the answer?". Here it removes a whole dimension from the DP. Always test the $k + 1$ case separately.

Solution code (as submitted)

```
/**
 * 2472. Maximum Number of Non-overlapping Palindrome Substrings
 * https://leetcode.com/problems/maximum-number-of-non-overlapping-palindrome-substrings/
 *
 * Key observation: if a palindrome of length  $\geq k$  is selected, removing one character from
 * each end keeps it a palindrome and keeps it inside the same span. So we can always shrink
 * a chosen palindrome to length  $k$  or  $k + 1$  without breaking anything. Therefore only
 * palindromes of length exactly  $k$  or  $k + 1$  ever need to be considered.
 *
 * dp[i] = max number of palindromes selectable within s[0..i].
 * dp[i] = max(dp[i-1], dp[i-k] + 1 if s[i-k..i) is a palindrome,
 *            dp[i-k-1] + 1 if s[i-k-1..i) is a palindrome)
 *
 * @param {string} s
 * @param {number} k
 * @return {number}
 */
var maxPalindromes = function (s, k) {
  const n = s.length;
  const isPalindrome = (lo, hi) => {
    // checks s[lo..hi)
```

```

    for (hi--; lo < hi; lo++, hi--) if (s[lo] !== s[hi]) return false;
    return true;
};

const dp = new Array(n + 1).fill(0);
for (let i = 1; i <= n; i++) {
    dp[i] = dp[i - 1];
    if (i >= k && isPalindrome(i - k, i)) dp[i] = Math.max(dp[i], dp[i - k] + 1);
    if (i >= k + 1 && isPalindrome(i - k - 1, i)) dp[i] = Math.max(dp[i], dp[i - k - 1] + 1);
}
return dp[n];
};

```

4.16 1621. Number of Sets of K Non-Overlapping Line Segments

Field	Value
Daily Challenge date	2026-09-16
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Math, Dynamic Programming, Combinatorics, Prefix Sum
Link	https://leetcode.com/problems/number-of-sets-of-k-non-overlapping-line-segments/
Result	Accepted, 68/68 tests, 0 ms, 58.4 MB (submission 2156489328)

What it asks (own words)

Points sit at $x = 0, 1, \dots, n-1$. Draw exactly k segments, each between two distinct points. Segments may not overlap but may share an endpoint. Count the ways modulo 10^9+7 .

Key constraints

- $n \leq 1000$, $k \leq n-1 \rightarrow$ an $O(n \cdot k)$ DP ($5 \cdot 10^5$) is fine; a closed formula is even better.

Reasoning

DP view. $dp[i][j]$ = ways to draw j segments using points $0..i$. Either no segment ends at i ($dp[i-1][j]$), or the last segment is $[a, i]$ for some $a < i$, leaving $j-1$ segments within $0..a$: $\sum_{a < i} dp[a][j-1]$. A running prefix sum makes each transition $O(1)$. This is the independent oracle in the tests.

Combinatorial view (the submitted solution). Written left to right, the endpoints are $a_1 < b_1 \leq a_2 < b_2 \leq \dots \leq a_k < b_k$. The $\leq$ (shared endpoints) is what makes direct counting awkward. Fix it by inserting one extra point in each of the $k-1$ gaps between consecutive segments, i.e. map $a_i \rightarrow a_i + (i-1)$ and $b_i \rightarrow b_i + (i-1)$. Now all $2k$ endpoints are strictly increasing within $0..n+k-2$, a set of $n+k-1$ points, and any $2k$ distinct points map back to a valid drawing. So the answer is $C(n+k-1, 2k)$.

Algorithm

Compute $C(n+k-1, 2k) \bmod p$ as $(\prod (top - i)) \cdot (2k)!^{-1}$, with the inverse from Fermat's little theorem $x^{-1} \equiv x^{(p-2)} \pmod p$ via fast exponentiation.

Why it works

The shift map is a bijection between valid drawings and $2k$ -subsets of $n+k-1$ points (checked exhaustively for all $n \leq 9$, and against the DP up to $n = 1000$).

JavaScript implementation details: BigInt is required here

- A product of two residues can be as large as $(10^9+7)^2 \approx 10^{18}$, well above $2^{53} \approx 9 \cdot 10^{15}$. With plain numbers the low digits are silently lost and the modulo result is wrong. So all modular multiplication uses **BigInt** (`1_000_000_007n`, `%`, `*`, `>=>`, `&`).
- Convert back with `Number(...)` at the end; the result is $< p$, so it's exact.
- The DP oracle only *adds* residues ($< 2 \cdot 10^9$), so it stays with plain numbers safely.

Edge cases

- $k = n-1$: every unit segment $[i, i+1]$ is used, which is the only way. $C(2n-2, 2n-2) = 1$.
- $k = 1$: $C(n, 2)$, i.e. choose any two points.

Bugs / debugging

- My first draft of the DP oracle tracked "open/closed segment" states and was convoluted enough that I didn't trust it. An oracle you can't trust defeats its purpose. I replaced it, **before running anything**, with the textbook prefix-sum DP, which is short enough to verify by eye. The submitted solution itself was unchanged.

Alternatives considered

- The prefix-sum DP ($O(n \cdot k)$) as a solution: perfectly acceptable, and it needs no modular inverse.
- Pascal's triangle up to row 2000: $O(n^2)$ memory/time, avoiding inverses.

Complexity

- Time: $O(k + \log p)$.
- Space: $O(1)$.

Reusable pattern

Stars and bars / "insert gaps" bijection. When items must be non-decreasing, shifting the i -th item by i turns " $\leq$ " into " $<$ ", and the count becomes a single binomial coefficient.

What to take away personally

In JS, any modular *multiplication* with a $\sim 10^9$ modulus needs BigInt (or a split multiplication). Addition is safe; multiplication is not.

Solution code (as submitted)

```
/**
 * 1621. Number of Sets of K Non-Overlapping Line Segments
 * https://leetcode.com/problems/number-of-sets-of-k-non-overlapping-line-segments/
 *
 * Bijection: segments may share endpoints, which makes direct counting awkward. Insert
 * k - 1 extra points so that every place where one segment could end and the next begin
 * gets its own point. Then a valid drawing of k segments on n points corresponds exactly to
 * choosing 2k distinct points out of n + k - 1 (pair them up left to right).
 * Answer:  $C(n + k - 1, 2k) \bmod 1e9+7$ .
 *
 * Products of two residues can reach  $\sim 1e18 > 2^{53}$ , so the arithmetic uses BigInt.
 *
 * @param {number} n
 * @param {number} k
 * @return {number}
 */
var numberOfSets = function (n, k) {
  const MOD = 1_000_000_007n;
  const top = BigInt(n + k - 1);
  const r = BigInt(2 * k);

  const modPow = (base, exp) => {
    let result = 1n;
    base %= MOD;
    while (exp > 0n) {

```

```

        if (exp & 1n) result = (result * base) % MOD;
        base = (base * base) % MOD;
        exp >>= 1n;
    }
    return result;
};

// C(top, r) = top! / (r! (top - r)!), computed as a product of r terms over r!
let numerator = 1n;
let denominator = 1n;
for (let i = 0n; i < r; i++) {
    numerator = (numerator * (top - i)) % MOD;
    denominator = (denominator * (i + 1n)) % MOD;
}
// MOD is prime, so the inverse of the denominator is denominator^(MOD - 2) (Fermat)
return Number((numerator * modPow(denominator, MOD - 2n)) % MOD);
};

```

4.17 1477. Find Two Non-overlapping Sub-arrays Each With Target Sum

Field	Value
Daily Challenge date	2026-09-17
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Hash Table, Binary Search, Dynamic Programming, Sliding Window
Link	https://leetcode.com/problems/find-two-non-overlapping-sub-arrays-each-with-target-sum/
Result	Accepted, 65/65 tests, 13 ms, 65.3 MB (submission 2156489733)

What it asks (own words)

Find two disjoint contiguous pieces of the array that each add up to `target`, minimizing their combined length. Return that minimum, or `-1` if two such pieces don't exist.

Key constraints

- $n \leq 10^5 \rightarrow$ we need $O(n)$ or $O(n \log n)$.
- All values are positive (≥ 1).** This is what makes a sliding window valid: extending the window only increases the sum, and shrinking only decreases it.

Reasoning

Two sub-problems:

- Find every subarray with sum = target.** With positive values, for each right end there is at most one left end that works, and it only moves right as the right end moves right. That's the classic two-pointer window.
- Pair them optimally without overlap.** Sweep left to right. When a window `[left..right]` hits the target, the best partner is the *shortest* target-sum subarray that ends **before** `left`. Keep a running array `bestEndingBy[i]` = the shortest such subarray ending at or before `i`, and read `bestEndingBy[left - 1]`.

Algorithm

For each `right`: add `arr[right]`; while `sum > target`, drop `arr[left++]`. Start with `best = bestEndingBy[right - 1]`. If `sum === target`: `len = right - left + 1`; if `bestEndingBy[left - 1]` is finite, update `answer` with it plus `len`; `best = min(best, len)`. Store `bestEndingBy[right] = best`. Return `answer` or `-1`.

Why it works

Every target-sum subarray is visited exactly once, at its right end. For the optimal pair (A before B), when the sweep reaches B's right end, `bestEndingBy[B.left - 1]` is at most `|A|`, so the answer is found. It can never pair overlapping pieces, because it only looks strictly before `left`.

JavaScript implementation details

- `Infinity` as the "none yet" sentinel works naturally with `Math.min`, and `Infinity + x` stays `Infinity`. The code still checks explicitly before adding, for clarity.
- `arr[left++]` removes the element and advances in one expression. It's concise, but read it carefully.

Edge cases

- Only one qualifying subarray $\rightarrow -1$.
- `[1, 1]`, target 1 $\rightarrow$ two single elements $\rightarrow 2$.
- Many overlapping candidates with different lengths: the prefix-best array picks the shortest valid partner.

Bugs / debugging

None. It was checked against a brute force (enumerate all target-sum subarrays, try all disjoint pairs) on 1000 random arrays.

Alternatives considered

- Prefix sums + hash map: needed if values could be zero or negative (the sliding window breaks then). Same "best-so-far" pairing idea.
- Compute best-from-left and best-from-right arrays separately, then combine. Equivalent, but uses two passes.

Complexity

- Time: $O(n)$: each index enters and leaves the window once.
- Space: $O(n)$ for `bestEndingBy`.

Reusable pattern

"Two non-overlapping X" $\rightarrow$ **sweep with a running best-X-so-far on the left**. It reduces a pair search to a single pass. Combined here with **sliding window for positive-sum subarrays**.

What to take away personally

Check the sign constraint first: positive values $\rightarrow$ sliding window; arbitrary values $\rightarrow$ prefix sums with a hash map.

Solution code (as submitted)

```
/**
 * 1477. Find Two Non-overlapping Sub-arrays Each With Target Sum
 * https://leetcode.com/problems/find-two-non-overlapping-sub-arrays-each-with-target-sum/
 *
 * All values are positive, so a sliding window finds every subarray with sum == target.
 * bestEndingBy[i] = length of the shortest target-sum subarray that ends at or before i.
 * When a window [left..right] hits the target, pair it with bestEndingBy[left - 1]: the best
 * subarray that finishes strictly before this one starts.
 *
 * @param {number[]} arr
 * @param {number} target
 * @return {number}
 */
var minSumOfLengths = function (arr, target) {
  const n = arr.length;
  const bestEndingBy = new Array(n).fill(Infinity);
  let answer = Infinity;
  let left = 0;
  let sum = 0;

  for (let right = 0; right < n; right++) {
    sum += arr[right];
    while (sum > target) sum -= arr[left++];
  }
```

```

let best = right > 0 ? bestEndingBy[right - 1] : Infinity;
if (sum === target) {
  const len = right - left + 1;
  if (left > 0 && bestEndingBy[left - 1] !== Infinity) {
    answer = Math.min(answer, bestEndingBy[left - 1] + len);
  }
  best = Math.min(best, len);
}
bestEndingBy[right] = best;
}
return answer === Infinity ? -1 : answer;
};

```

4.18 1520. Maximum Number of Non-Overlapping Substrings

Field	Value
Daily Challenge date	2026-09-18
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Hard
Topics	Hash Table, String, Greedy, Sorting
Link	https://leetcode.com/problems/maximum-number-of-non-overlapping-substrings/
Result	Accepted, 285/285 tests, 18 ms, 60.9 MB (submission 2156489786)

What it asks (own words)

Choose as many disjoint substrings as possible where each substring is "closed": if it contains a letter, it contains every occurrence of that letter in the whole string. Among the maximum-count choices, return the one with the smallest total length (it is unique). The order of the output doesn't matter.

Key constraints

- $n \leq 10^5$, but only 26 distinct letters $\rightarrow$ at most 26 candidate substrings.

Reasoning

1. **Candidates.** A closed substring that starts at position i must contain all of $s[i]$, so for the *smallest* closed substring starting at $\text{first}[c]$: start with $[\text{first}[c], \text{last}[c]]$, and while scanning it, extend the end to $\text{last}[x]$ of any letter x found inside. If some letter inside first appears *before* the start, no closed substring starts here (it would have to extend left). Only starts at some $\text{first}[c]$ can be minimal, which gives at most 26 candidates, each found in $O(n)$.
2. **Structure.** Two closed substrings can never partially overlap. If they share a position with letter x , both contain all of x . With a bit more care this shows they are either disjoint or nested. So the candidates form a forest of nested intervals.
3. **Selection.** To maximize count and then minimize length, we want the innermost intervals. Scan the candidates by start. If one starts after the last kept interval ends, keep it. Otherwise it lies inside the last kept one, so **replace** that one (same count, shorter, and it ends no later).

Algorithm

Compute first/last per letter. For i in order where $i === \text{first}[s[i]]$: $\text{end} = \text{closedEnd}(i)$; skip if -1 ; if $i > \text{prevEnd}$, push $s.\text{slice}(i, \text{end}+1)$, otherwise overwrite the last element; set $\text{prevEnd} = \text{end}$.

Why it works

Among nested candidates, the innermost one is always at least as good (same contribution of 1, shorter, and it leaves more room). Among disjoint candidates, taking all of them is optimal. The replace-if-inside scan implements exactly "keep all innermost candidates in left-to-right order".

JavaScript implementation details

- `charCodeAt(i) - 97` for letter indices, with arrays of 26 instead of Maps.
- `s.slice(i, end + 1)` has an exclusive end, hence the `+ 1`.

- The tests normalize outputs with `[...arr].sort()` because any order is accepted.

Edge cases

- The whole string is the only closed substring (e.g. "abab") → one piece.
- All letters distinct → every letter alone.
- Nested candidates: "abba" → ["bb"] rather than ["abba"].

Bugs / debugging

None. It was verified against an exhaustive DP over all closed substrings (maximize count, then minimize total length) on 800 random strings over {a,b,c,d}.

Alternatives considered

- Collect the ≤ 26 candidates, sort by end, and greedily take the earliest-ending non-overlapping ones. This is the interval-scheduling view of the same idea.

Complexity

- Time: $O(26 \cdot n)$ in the worst case (each candidate scan is $O(n)$).
- Space: $O(26)$ plus the output.

Reusable pattern

Closure / expansion intervals + laminar (nested-or-disjoint) structure → greedy innermost selection. Also: when the alphabet is small, "one candidate per distinct letter" bounds the search.

What to take away personally

Proving that candidates are nested-or-disjoint is what makes the greedy safe. Look for that structural property before trusting a greedy on intervals.

Solution code (as submitted)

```
/**
 * 1520. Maximum Number of Non-Overlapping Substrings
 * https://leetcode.com/problems/maximum-number-of-non-overlapping-substrings/
 *
 * A valid substring must be "closed": every character inside it has all of its occurrences
 * inside it. For each character c, the smallest closed substring starting at first[c] is
 * found by repeatedly extending the end to cover last[x] of every character x inside it.
 * If some x inside has first[x] < first[c], no closed substring starts there.
 *
 * Any two such candidates are either disjoint or nested (they can't partially overlap, since
 * each one contains all occurrences of every character it touches). Scanning candidates by
 * start: if a candidate starts after the previously kept one ends, keep it as a new piece;
 * otherwise it lies inside the previous one, so replace that one with this smaller one
 * (same count, less total length, and it frees up room to the right).
 *
 * @param {string} s
 * @return {string[]}
 */
var maxNumOfSubstrings = function (s) {
  const n = s.length;
  const first = new Array(26).fill(-1);
```

```

const last = new Array(26).fill(-1);
for (let i = 0; i < n; i++) {
  const c = s.charCodeAt(i) - 97;
  if (first[c] === -1) first[c] = i;
  last[c] = i;
}

// returns the end of the smallest closed substring starting at `start`, or -1 if none
const closedEnd = (start) => {
  let end = last[s.charCodeAt(start) - 97];
  for (let i = start; i <= end; i++) {
    const c = s.charCodeAt(i) - 97;
    if (first[c] < start) return -1;
    if (last[c] > end) end = last[c];
  }
  return end;
};

const result = [];
let prevEnd = -1;
for (let i = 0; i < n; i++) {
  if (i !== first[s.charCodeAt(i) - 97]) continue;
  const end = closedEnd(i);
  if (end === -1) continue;
  if (i > prevEnd) result.push(s.slice(i, end + 1));
  else result[result.length - 1] = s.slice(i, end + 1);
  prevEnd = end;
}
return result;
};

```

4.19 1401. Circle and Rectangle Overlapping

Field	Value
Daily Challenge date	2026-09-19
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Math, Geometry
Link	https://leetcode.com/problems/circle-and-rectangle-overlapping/
Result	Accepted, 90/90 tests, 0 ms, 52.6 MB (submission 2156489834)

What it asks (own words)

Does a circle (radius, centre) share at least one point with an axis-aligned rectangle? Touching counts.

Key constraints

- All inputs are integers, the radius is ≤ 2000 , coordinates are within $\pm 10^4$. Squared distances stay below about 10^9 , which is exact in JS.

Reasoning

The shapes intersect iff the rectangle point **closest to the circle's centre** lies within the radius. For an axis-aligned rectangle the closest point is easy: clamp the centre's x into `[x1, x2]` and its y into `[y1, y2]` independently. If the centre is inside the rectangle, the clamp returns the centre itself (distance 0).

Algorithm

```
dx = clamp(xC, x1, x2) - xC, dy = clamp(yC, y1, y2) - yC; return dx2 + dy2 ≤ r2.
```

Why it works

The squared distance from the centre to a rectangle point is $(x - xC)^2 + (y - yC)^2$, and the two terms can be minimized separately over the independent ranges `[x1, x2]` and `[y1, y2]`. Each is minimized at the clamp. Comparing *squared* distances avoids `Math.sqrt` and keeps everything in exact integer arithmetic.

JavaScript implementation details

- `clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v))` is a reusable idiom.
- Avoid `Math.hypot` / `Math.sqrt` for boundary comparisons: float rounding can flip an exact tangency.

Edge cases

- Tangent (touching at exactly one point) → true ($\leq$).
- Near a corner: the corner is the closest point (distance $\sqrt{2}$ vs radius 1 or 2).
- Circle entirely inside the rectangle, or rectangle entirely inside the circle → true.

Bugs / debugging

None. It was verified against a point-sampling oracle (quarter-unit grid over the rectangle; exact for integer inputs because the closest point is a lattice point) on 1000 random cases.

Alternatives considered

- Case analysis (centre inside; circle crosses an edge; circle contains a corner). It's error-prone, and the clamp handles every case uniformly.

Complexity

- Time: $O(1)$.
- Space: $O(1)$.

Reusable pattern

Closest point on an axis-aligned box = per-axis clamp. It extends to 3-D boxes and to "distance from a point to a segment", where you clamp the projection parameter.

What to take away personally

Compare squared distances whenever possible, since exact integer maths beats floating-point square roots. And compare with 836 (rectangle–rectangle): both reduce 2-D geometry to independent per-axis checks.

Solution code (as submitted)

```
/**
 * 1401. Circle and Rectangle Overlapping
 * https://leetcode.com/problems/circle-and-rectangle-overlapping/
 *
 * The rectangle point closest to the circle's centre is found by clamping the centre into
 * the rectangle on each axis independently. The shapes share a point iff that closest point
 * is within the radius. Compare squared distances to stay in exact integer arithmetic.
 *
 * @param {number} radius
 * @param {number} xCenter
 * @param {number} yCenter
 * @param {number} x1
 * @param {number} y1
 * @param {number} x2
 * @param {number} y2
 * @return {boolean}
 */
var checkOverlap = function (radius, xCenter, yCenter, x1, y1, x2, y2) {
  const clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v));
  const dx = clamp(xCenter, x1, x2) - xCenter;
  const dy = clamp(yCenter, y1, y2) - yCenter;
  return dx * dx + dy * dy <= radius * radius;
};
```

4.20 3498. Reverse Degree of a String

Field	Value
Daily Challenge date	2026-09-20
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Easy
Topics	String, Simulation
Link	https://leetcode.com/problems/reverse-degree-of-a-string/
Result	Accepted, 933/933 tests, 1 ms, 54.8 MB (submission 2156490211)

What it asks (own words)

Give each letter a value from the *reversed* alphabet ('a' = 26 ... 'z' = 1), multiply by its 1-based position in the string, and sum.

Key constraints

- Length ≤ 1000 , so the maximum is $26 \cdot (1 + \dots + 1000) = 13,013,000$, which is tiny.

Reasoning

This is pure simulation. The only detail is the mapping: with `code = charCode - 97` ($a \rightarrow 0, \dots, z \rightarrow 25$), the reversed rank is `26 - code`.

Algorithm

Sum `(26 - (charCode - 97)) · (i + 1)` over all i .

Why it works

It's the definition, computed directly.

JavaScript implementation details

- `charCodeAt(i) - 97` is the standard letter-to-index idiom ('a' is 97 in ASCII/UTF-16).
- Mind the 1-based position `(i + 1)` versus the 0-based loop index.

Edge cases

- Single 'a' $\rightarrow 26$; single 'z' $\rightarrow 1$.
- Maximum-length string of 'a' $\rightarrow 26 \cdot 500,500$ (tested).

Bugs / debugging

None.

Alternatives considered

None needed.

Complexity

- Time: $O(n)$.

- Space: $O(1)$.

Reusable pattern

Character arithmetic via char codes. Mapping letters to 0..25 (or reversed, 25..0) comes up constantly: counting arrays, Caesar shifts, per-letter state (see 940, 1520 this month).

What to take away personally

Off-by-one hygiene: write down both index conventions (0-based loop, 1-based position) before coding.

Solution code (as submitted)

```
/**
 * 3498. Reverse Degree of a String
 * https://leetcode.com/problems/reverse-degree-of-a-string/
 *
 * Direct simulation: 'a' is worth 26 ... 'z' is worth 1, multiplied by the 1-based position.
 *
 * @param {string} s
 * @return {number}
 */
var reverseDegree = function (s) {
  let total = 0;
  for (let i = 0; i < s.length; i++) {
    const reversedRank = 26 - (s.charCodeAt(i) - 97); // 'a' -> 26, 'z' -> 1
    total += reversedRank * (i + 1);
  }
  return total;
};
```

4.21 3524. Find X Value of Array I

Field	Value
Daily Challenge date	2026-09-21
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Math, Dynamic Programming
Link	https://leetcode.com/problems/find-x-value-of-array-i/
Result	Accepted, 782/782 tests, 48 ms, 73.5 MB (submission 2156490267)

What it asks (own words)

Chop off any prefix and any suffix (possibly empty) as long as something remains. The remainder is a non-empty subarray. For each residue x in $0..k-1$, count how many subarrays have a product $\equiv x \pmod k$.

Key constraints

- $n \leq 10^5 \rightarrow O(n^2)$ subarrays ($5 \cdot 10^9$) is too many to enumerate.
- $k \leq 5$, so residues are a tiny state.
- Values up to 10^9 : take `% k` first, so products stay tiny.

Reasoning

Products mod k are *composable*: $(a \cdot b) \bmod k = ((a \bmod k) \cdot (b \bmod k)) \bmod k$. So, for subarrays **ending at index j** , we only need a histogram of their residues. Moving to $j+1$ with value v :

- every subarray ending at j extends to $j+1$: residue $r \rightarrow r \cdot v \bmod k$;
- plus the new single-element subarray $[v]$ with residue $v \bmod k$.

Add each index's histogram into the global answer.

Algorithm

```
endingHere = [0]*k, result = [0]*k. For each v: next[(r·v) % k] += endingHere[r] for all r; next[v % k] += 1; add next into result; endingHere = next.
```

Why it works

Every non-empty subarray has a unique right end, so it's counted exactly once, in that index's histogram. The transition is exact by the modular product rule.

JavaScript implementation details

- Reduce each value `% k` before multiplying: $r \cdot v < 25$. That's exact, with no overflow concerns.
- The counts can reach $n(n+1)/2 \approx 5 \cdot 10^9$. That exceeds 32-bit ints but is far below 2^{53} , so plain JS numbers are exact. A test checks this at $n = 10^5$. (In Java/C++ this would need a `long`.)
- $k = 1$: everything is residue 0; `value % 1 === 0` handles it naturally.

Edge cases

- $k = 1 \rightarrow [n(n+1)/2]$.

- Values that are multiples of k make every subarray containing them residue 0.

Bugs / debugging

None. It was checked against the $O(n^2)$ brute force on 500 random arrays with values up to 10^9 .

Alternatives considered

- Prefix products with modular inverses. This doesn't work in general: $\text{mod } k$ isn't prime and zero residues have no inverse.

Complexity

- Time: $O(n \cdot k)$.
- Space: $O(k)$.

Reusable pattern

"Count subarrays by some composable property" → DP over subarrays ending at i , keyed by the property's (small) state. The same shape works for subarray XOR, sum mod k , bitwise-AND values, etc.

What to take away personally

When the modulus or state space is tiny, keep a histogram per position instead of enumerating subarrays. Part II (3525) makes this updatable with a segment tree.

Solution code (as submitted)

```
/**
 * 3524. Find X Value of Array I
 * https://leetcode.com/problems/find-x-value-of-array-i/
 *
 * Removing a prefix and a suffix leaves a non-empty subarray, so we must count subarrays by
 * (product mod k).  $k \leq 5$ , so track, for subarrays ENDING at the current index, how many
 * have each residue. Extending to the next value  $v$ : every old residue  $r$  becomes  $r \cdot v \bmod k$ ,
 * plus the new single-element subarray  $[v]$ .
 *
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
var resultArray = function (nums, k) {
  const result = new Array(k).fill(0);
  let endingHere = new Array(k).fill(0);
  for (const value of nums) {
    const v = value % k;
    const next = new Array(k).fill(0);
    for (let r = 0; r < k; r++) next[(r * v) % k] += endingHere[r];
    next[v] += 1;
    for (let r = 0; r < k; r++) result[r] += next[r];
    endingHere = next;
  }
  return result;
};
```

4.22 3525. Find X Value of Array II

Field	Value
Daily Challenge date	2026-09-22
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Hard
Topics	Array, Math, Segment Tree
Link	https://leetcode.com/problems/find-x-value-of-array-ii/
Result	Accepted, 783/783 tests, 286 ms, 103 MB (submission 2156490309)

What it asks (own words)

Process queries in order. Each query permanently sets `nums[index] = value`, then (temporarily) looks at the array from `start` onwards and asks: how many **non-empty prefixes** of that part have product $\equiv x \pmod k$?

Key constraints

- $n \leq 10^5$, $q \leq 2 \cdot 10^4 \rightarrow O(n \cdot q) = 2 \cdot 10^9$ is too slow. We need about $O(\log n)$ per query and update.
- $k \leq 5$, a tiny residue space again (compare 3524).

Reasoning

We need a structure that supports **point update** and **range query on a suffix**, where the query result (a histogram of prefix-product residues) can be **merged** from pieces. That's what a segment tree does, provided we can define the merge.

For a segment, store:

- `prod` = product of the segment mod k ;
- `cnt[r]` = number of non-empty prefixes of the segment with product $\equiv r$.

Merge L then R: prefixes of the combined segment are either prefixes of L (`L.cnt` as is), or all of L followed by a non-empty prefix of R: residue `L.prod · r`. So `cnt[(L.prod·r) % k] += R.cnt[r]`, and `prod = L.prod · R.prod`. The merge is associative, which a segment tree requires, and it is **not commutative**, so order matters.

Algorithm

- Build the tree over `nums` (leaf: `prod = v % k`, `cnt[v % k] = 1`).
- For each query: point-update `index`, then query `[start, n-1]` by visiting covering nodes **left to right**, folding each into an accumulator (`accProd`, `accCnt`) with the same merge rule. Answer `accCnt[x]`.

Why it works

The segment-tree decomposition of `[start, n-1]` is a left-to-right sequence of $O(\log n)$ disjoint nodes whose concatenation is exactly that range. Folding them in order with an associative merge gives the same result as merging element by element.

JavaScript implementation details

- Flat typed arrays: `prod` is an `Int32Array(4n)` and `cnt` is an `Int32Array(4n · k)` indexed `node * k + r`. This avoids 400k small JS arrays (memory and GC pressure).

- `cnt.fill(0, from, to)` resets just one node's slice.
- The recursive query visits the left child before the right. That ordering is what makes the non-commutative merge correct; swapping them would be a subtle bug.
- `accProd` starts at `1 % k` (which is 0 when $k = 1$), so the $k = 1$ case needs no special handling.

Edge cases

- `start = n - 1` → a single-element range.
- $k = 1$ → every prefix counts for $x = 0$.
- Updates persist across queries (the brute-force reference also mutates its copy).

Bugs / debugging

None on submission. The main risk, merge order, was addressed in the design and verified against a brute force that applies each update and scans prefixes directly (400 random cases), plus a full-size timing test.

Optimization opportunity

286 ms / 103 MB passed comfortably. An iterative bottom-up tree would cut recursion overhead, and since queries are always suffixes, a specialised structure could avoid the generic range walk.

Alternatives considered

- Recomputing from `start` each query: $O(n)$ per query, $2 \cdot 10^9$ total, too slow.
- Sqrt decomposition with per-block (prod, cnt): $O(\sqrt{n} \cdot k)$ per query, simpler but slower.

Complexity

- Build: $O(n \cdot k)$. Each update and query: $O(k \log n)$.
- Space: $O(n \cdot k)$.

Reusable pattern

Segment tree with a custom, associative (possibly non-commutative) monoid. Define what a node stores, prove the merge is associative, and remember to merge in order. The same framework covers max-subarray-sum nodes, matrix products, string hashes, etc.

What to take away personally

Part I's per-index histogram becomes Part II's node summary. When a Part II adds updates, ask "what summary of a segment lets me glue two segments together?"

Solution code (as submitted)

```
/**
 * 3525. Find X Value of Array II
 * https://leetcode.com/problems/find-x-value-of-array-ii/
 *
 * Each query asks: after a point update, for the array starting at `start`, how many
 * non-empty prefixes nums[start..j] have product == x (mod k)?
 *
 * Segment tree. Each node over a range stores:
 *   prod    = product of the range mod k
 *   cnt[r]  = number of non-empty prefixes of the range whose product mod k is r
```

```

* Merging left segment L with right segment R (L first):
*   prod = L.prod * R.prod
*   cnt  = L.cnt, plus for each r: every prefix of R extended by all of L -> L.prod * r
* k <= 5, so a merge is O(k). Queries on [start, n-1] merge O(log n) nodes left to right.
*
* @param {number[]} nums
* @param {number} k
* @param {number[][]} queries
* @return {number[]}
*/
var resultArray = function (nums, k, queries) {
  const n = nums.length;
  const size = 4 * n;
  const prod = new Int32Array(size);
  const cnt = new Int32Array(size * k); // cnt[node * k + r]

  const setLeaf = (node, value) => {
    const v = value % k;
    prod[node] = v;
    cnt.fill(0, node * k, node * k + k);
    cnt[node * k + v] = 1;
  };

  const pull = (node) => {
    const L = node * 2;
    const R = node * 2 + 1;
    const lp = prod[L];
    prod[node] = (lp * prod[R]) % k;
    for (let r = 0; r < k; r++) cnt[node * k + r] = cnt[L * k + r];
    for (let r = 0; r < k; r++) cnt[node * k + ((lp * r) % k)] += cnt[R * k + r];
  };

  const build = (node, lo, hi) => {
    if (lo === hi) return setLeaf(node, nums[lo]);
    const mid = (lo + hi) >> 1;
    build(node * 2, lo, mid);
    build(node * 2 + 1, mid + 1, hi);
    pull(node);
  };

  const update = (node, lo, hi, idx, value) => {
    if (lo === hi) return setLeaf(node, value);
    const mid = (lo + hi) >> 1;
    if (idx <= mid) update(node * 2, lo, mid, idx, value);
    else update(node * 2 + 1, mid + 1, hi, idx, value);
    pull(node);
  };

  // accumulator for a left-to-right query: product so far and prefix counts so far
  let accProd = 1 % k;
  const accCnt = new Array(k).fill(0);
  const absorb = (node) => {
    for (let r = 0; r < k; r++) accCnt[(accProd * r) % k] += cnt[node * k + r];
    accProd = (accProd * prod[node]) % k;
  };

  const query = (node, lo, hi, from) => {
    if (hi < from) return;
    if (lo >= from) return absorb(node);
    const mid = (lo + hi) >> 1;
    query(node * 2, lo, mid, from); // left part first keeps the order correct
    query(node * 2 + 1, mid + 1, hi, from);
  };

```

```
};

build(1, 0, n - 1);
const result = [];
for (const [index, value, start, x] of queries) {
    update(1, 0, n - 1, index, value);
    accProd = 1 % k;
    accCnt.fill(0);
    query(1, 0, n - 1, start);
    result.push(accCnt[x]);
}
return result;
};
```

4.23 1658. Minimum Operations to Reduce X to Zero

Field	Value
Daily Challenge date	2026-09-23
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Hash Table, Binary Search, Sliding Window, Prefix Sum
Link	https://leetcode.com/problems/minimum-operations-to-reduce-x-to-zero/
Result	Accepted, 97/97 tests, 7 ms, 66.7 MB (submission 2156490604)

What it asks (own words)

Repeatedly take an element from either end of the array and subtract it from x . What's the fewest removals that bring x to exactly 0? Return -1 if impossible.

Key constraints

- $n \leq 10^5$; values are **positive** ($1..10^4$); x up to 10^9 .

Reasoning

Searching over sequences of left/right choices is exponential, and even "i from the left, j from the right" is $O(n^2)$.

Flip the problem: whatever you remove from both ends, what *remains* is one contiguous middle block. Removed sum = $x \Leftrightarrow$ remaining sum = $\text{total} - x$. Fewest removals $\Leftrightarrow$ **longest** middle block with sum exactly $\text{total} - x$. With positive values that's a sliding-window problem (see 1477).

Algorithm

- keep = $\text{total} - x$. If $\text{keep} < 0 \rightarrow -1$. If $\text{keep} === 0 \rightarrow n$ (remove everything).
- Sliding window for the longest subarray with sum `keep`.
- Return $n - \text{longest}$, or -1 if none exists.

Why it works

There's a bijection between (left count, right count) choices and middle blocks, and the number of removals is $n - (\text{block length})$. With positive values, the window finds every exact-sum subarray's maximal extent.

JavaScript implementation details

- The `keep === 0` special case matters: the window loop would never record an empty block, since it only checks after adding an element.
- $\text{total} \leq 10^9$ fits easily in a JS number.

Edge cases

- $x > \text{total} \rightarrow -1$.
- $x === \text{total} \rightarrow n$.
- Taking only from one side (e.g. just the rightmost element).

Bugs / debugging

None. It was verified against a brute force over all (left, right) split sizes on 1000 random arrays. That brute force also confirms the complement argument independently.

Alternatives considered

- Prefix sums from the left in a hash map, then scanning suffix sums from the right and looking up `x - suffix`. Also $O(n)$, and it's the variant to use if values could be non-positive.

Complexity

- Time: $O(n)$.
- Space: $O(1)$.

Reusable pattern

Complement transformation: "remove from both ends" $\Leftrightarrow$ "keep a middle subarray". Minimizing what you remove is maximizing what you keep.

What to take away personally

When choices come from both ends, look at what's left in the middle. Compare with 1477: same sliding-window engine, different wrapper.

Solution code (as submitted)

```
/**
 * 1658. Minimum Operations to Reduce X to Zero
 * https://leetcode.com/problems/minimum-operations-to-reduce-x-to-zero/
 *
 * Removing some elements from the left and some from the right leaves a contiguous middle
 * block. Removed sum == x <=> kept middle sum == total - x. Minimizing removals means
 * maximizing the kept block, and with all values positive a sliding window finds the
 * longest subarray with an exact sum.
 *
 * @param {number[]} nums
 * @param {number} x
 * @return {number}
 */
var minOperations = function (nums, x) {
  const n = nums.length;
  let total = 0;
  for (const v of nums) total += v;
  const keep = total - x;
  if (keep < 0) return -1; // even removing everything is not enough
  if (keep === 0) return n; // remove everything

  let longest = -1;
  let left = 0;
  let sum = 0;
  for (let right = 0; right < n; right++) {
    sum += nums[right];
    while (sum > keep) sum -= nums[left++];
    if (sum === keep) longest = Math.max(longest, right - left + 1);
  }
  return longest === -1 ? -1 : n - longest;
};
```

4.24 3550. Smallest Index With Digit Sum Equal to Index

Field	Value
Daily Challenge date	2026-09-24
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Easy
Topics	Array, Math
Link	https://leetcode.com/problems/smallest-index-with-digit-sum-equal-to-index/
Result	Accepted, 982/982 tests, 1 ms, 55 MB (submission 2156490663)

What it asks (own words)

Find the first position i where the digits of `nums[i]` add up to i . Return -1 if there's none.

Key constraints

- $n \leq 100$, values $0..1000 \rightarrow$ digit sums ≤ 27 (for 999). So only indices $0..27$ can ever match.

Reasoning

This is a linear scan with a digit-sum helper. The observation that digit sums are at most 27 means you could even stop at index 27; not needed at this size, but a useful habit.

Algorithm

For $i = 0..n-1$: if `digitSum(nums[i]) === i`, return i . Return -1 .

Why it works

It evaluates the condition in index order, so the first match is the smallest.

JavaScript implementation details

- `v % 10` gives the last digit; `Math.floor(v / 10)` drops it. JS `/` is float division, so the floor is required, unlike `//` in Python or integer `/` in Java.
- `digitSum(0)` returns 0 because the loop body never runs. Correct, and tested.

Edge cases

- `[0] → 0`.
- Large index with a small value: index ≥ 28 can never match.

Bugs / debugging

None. One of my own test lines was written confusingly (a leftover ternary) and was rewritten to a plain expected value before running. Clear tests matter as much as clear code.

Alternatives considered

- `String(v).split('').reduce(...)`: works, but allocates.

Complexity

- Time: $O(n \cdot d)$, where $d \leq 4$ digits.
- Space: $O(1)$.

Reusable pattern

Digit extraction loop ($\% 10$, $/ 10$), the building block for digit sums, palindromic numbers, and digit DP.

What to take away personally

Bound the possible values (digit sum ≤ 27) because it often shows which inputs can be skipped entirely.

Solution code (as submitted)

```
/**
 * 3550. Smallest Index With Digit Sum Equal to Index
 * https://leetcode.com/problems/smallest-index-with-digit-sum-equal-to-index/
 *
 * Scan left to right, computing each digit sum with % 10 and integer division.
 *
 * @param {number[]} nums
 * @return {number}
 */
var smallestIndex = function (nums) {
  const digitSum = (v) => {
    let sum = 0;
    for (; v > 0; v = Math.floor(v / 10)) sum += v % 10;
    return sum;
  };
  for (let i = 0; i < nums.length; i++) {
    if (digitSum(nums[i]) === i) return i;
  }
  return -1;
};
```

4.25 1096. Brace Expansion II

Field	Value
Daily Challenge date	2026-09-25
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Hard
Topics	Hash Table, String, Backtracking, Stack, Breadth-First Search, Sorting
Link	https://leetcode.com/problems/brace-expansion-ii/
Result	Accepted, 115/115 tests, 6 ms, 60.4 MB (submission 2156490708)

What it asks (own words)

A mini-language describes sets of words: a letter is itself; `{A,B,...}` is the union of its parts; writing expressions next to each other concatenates every word of the first with every word of the second. Expand the expression and return the distinct words in sorted order.

Key constraints

- Expression length ≤ 60 , so output sizes stay manageable. The challenge is parsing correctly, not performance.

Reasoning

This is a **grammar with two operators of different precedence**, like arithmetic:

- concatenation (implicit, like `×`) binds tighter;
- comma (union, like `+`) binds looser, and only appears inside braces.

The classic technique is a **recursive-descent parser**, with one function per precedence level:

```
union := concat (',' concat)*
concat := term term*           (stops at ',' or '}' or end)
term   := letter | '{' union '}'
```

Each function consumes characters from a shared position `i` and returns a `Set` of words. Sets deduplicate for free.

Algorithm

- `parseUnion`: parse one concat, then while the next char is `,`, skip it and union in another concat.
- `parseConcat`: start with `{ '' }` (the identity for concatenation); for each term, replace the set with the cartesian product `{a + b}`.
- `term`: a letter gives `{letter}`; a `{` means skip, recurse into `parseUnion`, then skip `}`.
- Top level: `parseUnion()`, then spread into an array and sort.

Why it works

Each grammar rule maps directly to one function, and the stop conditions in `parseConcat` (`,` or `}`) implement precedence: a comma always ends the current concatenation and hands control back to the union level. Nested braces recurse naturally.

JavaScript implementation details

- **Closures over a shared index** `let i = 0` keep the parser compact. Every function advances the same cursor.
- `new Set([''])` as the starting value makes the first product just the term itself.
- `[...set].sort()`: the default sort is lexicographic by UTF-16 code units, which is correct for lowercase words. (For numbers you'd need a comparator.)
- `parseUnion` and `parseConcat` are mutually recursive `const` arrow functions. That works because each is only *called* after both are defined.

Edge cases

- Plain letters `"abc"` → one word, via concatenation of three terms.
- Duplicates `{a,a}{a,a} → ["aa"]`.
- Deep nesting `{{{a}}}`.
- A mix of bare letters and groups: `a{b,c}` inside a union.

Bugs / debugging

None. It was tested with all grammar examples from the statement plus the edge cases above. There's no simple independent oracle for this one; the statement's own worked examples served as the specification tests.

Alternatives considered

- Two stacks (operands and operators), shunting-yard style. It works, but it's harder to get right than a parser that mirrors the grammar.
- BFS expanding the first innermost brace group repeatedly: simpler to write, but it can blow up with duplicates before deduplication.

Complexity

- Time: proportional to the total size of the intermediate sets, bounded by the output size times the expression length for this grammar.
- Space: the same order, for the sets.

Reusable pattern

Recursive descent: one function per precedence level, sharing a cursor. It handles calculators, JSON-like formats, template languages, and this problem.

What to take away personally

When a problem gives you a *grammar*, write the grammar first and then translate each rule into a function. The code almost writes itself.

Solution code (as submitted)

```
/**
 * 1096. Brace Expansion II
 * https://leetcode.com/problems/brace-expansion-ii/
 *
 * Recursive-descent parser for the grammar:
 * union := concat ('|' concat)*      -> set union
 * concat := term term*               -> cartesian product of word sets
 */
```

```

*   term   := letter | '{' union '}'
* A concat ends at ',' or '}' (or end of input). Sets de-duplicate automatically.
*
* @param {string} expression
* @return {string[]}
*/
var braceExpansionII = function (expression) {
  let i = 0;

  const parseUnion = () => {
    const result = new Set(parseConcat());
    while (expression[i] === ',') {
      i++; // skip ','
      for (const word of parseConcat()) result.add(word);
    }
    return result;
  };

  const parseConcat = () => {
    let words = new Set(['']);
    while (i < expression.length && expression[i] !== ',' && expression[i] !== '}') {
      let term;
      if (expression[i] === '{') {
        i++; // skip '{'
        term = parseUnion();
        i++; // skip '}'
      } else {
        term = new Set([expression[i]]);
        i++;
      }
      const combined = new Set();
      for (const left of words) for (const right of term) combined.add(left + right);
      words = combined;
    }
    return words;
  };

  return [...parseUnion()].sort();
};

```

4.26 1807. Evaluate the Bracket Pairs of a String

Field	Value
Daily Challenge date	2026-09-26
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	Array, Hash Table, String
Link	https://leetcode.com/problems/evaluate-the-bracket-pairs-of-a-string/
Result	Accepted, 105/105 tests, 59 ms, 94.9 MB (submission 2156490947)

What it asks (own words)

A template string contains (key) placeholders (never nested). Replace each with its value from a dictionary, or with ? if the key is unknown.

Key constraints

- $|s| \leq 10^5$ and up to 10^5 dictionary entries → a hash map for lookups, and linear-time output building.

Reasoning

Two classic performance traps:

1. Searching the knowledge list for each key would be $O(n \cdot m)$. A `Map` makes it $O(1)$.
2. Building the result with repeated `+=` on a string. Modern engines optimize this reasonably well, but collecting pieces in an array and calling `join` once is the reliably linear approach.

Algorithm

`values = new Map(knowledge)`. Scan `s`: copy normal characters; at `(`, find the matching `)` with `indexOf`, look up the key, push the value or `?`, and jump the index to the `)`.

Why it works

No nesting means the next `)` after a `(` always closes it. Each character is visited a constant number of times.

JavaScript implementation details

- `new Map(knowledge)` works directly because `knowledge` is already an array of `[key, value]` pairs, the exact shape the `Map` constructor accepts.
- `values.get(key) ?? '?'` uses **nullish coalescing**, which falls back only on `undefined / null`. `||` would also replace an empty-string value; the values here are non-empty, but `??` states the intent precisely.
- `s.indexOf(')', i)` searches from position `i`, and setting `i = close` lets the loop's `i++` step past it.

Edge cases

- No brackets or empty knowledge.
- Keys match exactly: `(ab)` does not match key `a`.
- Values aren't re-evaluated even if they look like keys.

Bugs / debugging

None.

Alternatives considered

- `s.replace(/\((\w+)\)/g, (_, k) => values.get(k) ?? '?')`: a one-liner with a regex and a replacer function. It's idiomatic JS and also linear, but the manual scan shows the mechanics.

Complexity

- Time: $O(|s| + \text{total size of knowledge})$.
- Space: $O(|s| + |\text{knowledge}|)$.

Reusable pattern

Tokenize-and-substitute with a hash map. It's the core of every template engine. Use a Map for lookups and an array + `join` for output.

What to take away personally

Learn `??` versus `||`, and remember that `new Map(arrayOfPairs)` exists.

Solution code (as submitted)

```
/**
 * 1807. Evaluate the Bracket Pairs of a String
 * https://leetcode.com/problems/evaluate-the-bracket-pairs-of-a-string/
 *
 * Put the knowledge into a Map for O(1) lookups, then scan once: copy plain characters,
 * and when a '(' starts, read the key up to ')' and emit its value (or '?').
 * Pieces are collected in an array and joined once at the end.
 *
 * @param {string} s
 * @param {string[][]} knowledge
 * @return {string}
 */
var evaluate = function (s, knowledge) {
  const values = new Map(knowledge);
  const out = [];
  for (let i = 0; i < s.length; i++) {
    if (s[i] !== '(') {
      out.push(s[i]);
      continue;
    }
    const close = s.indexOf(')', i);
    const key = s.slice(i + 1, close);
    out.push(values.get(key) ?? '?');
    i = close; // loop increment moves past ')'
  }
  return out.join('');
};
```

4.27 1190. Reverse Substrings Between Each Pair of Parentheses

Field	Value
Daily Challenge date	2026-09-27
Solved	2026-09-28, after the daily window (solved problem, not a completed Daily Challenge)
Difficulty	Medium
Topics	String, Stack, Bracket Sequences
Link	https://leetcode.com/problems/reverse-substrings-between-each-pair-of-parentheses/
Result	Accepted, 40/40 tests, 1 ms, 52.8 MB (submission 2156491011)

What it asks (own words)

Reverse the text inside every pair of parentheses, innermost first, and then drop the parentheses.

Key constraints

- $|s| \leq 2000$, so even $O(n^2)$ passes. But there is a neat $O(n)$ method worth learning.

Reasoning

Straightforward ($O(n^2)$): keep a stack of partial strings. `(` pushes a new empty string, `)` pops, reverses and appends to the parent. Correct, but each reversal copies up to n characters. This is the test oracle.

Wormhole walk ($O(n)$): think about how the final string is *read*. Reversing a bracketed part means reading it right-to-left. Nested reversals flip direction again. So:

1. Pre-compute each bracket's partner with a stack.
2. Walk the string with a direction `dir = ±1`. Output letters as you pass them. On hitting a bracket, **teleport to its partner and flip direction**, then keep walking.

Entering a pair from either side lands you on its other end, heading back inward in the opposite direction, which is exactly reading the inside reversed. Leaving through the other bracket teleports you back out and flips again.

Algorithm

1. `partner[]` via a stack of `(` indices.
2. `for (i = 0, dir = 1; i < n; i += dir):` if `s[i]` is a bracket $\rightarrow$ `i = partner[i]`, `dir = -dir`; else output `s[i]`.
3. Join the output.

Why it works

Each bracket is visited exactly twice (once in each direction), and each letter exactly once. The number of direction flips before reaching a letter equals the number of pairs enclosing it, modulo 2, which matches how many times the stack approach would reverse it.

JavaScript implementation details

- `Int32Array` for `partner` (compact, zero-initialised).
- Two loop variables in one `for` header: `for (let i = 0, dir = 1; i < n; i += dir)`. The increment uses the *current* direction, so after a teleport `i += dir` moves inward from the partner.

- Tests generate random balanced strings to compare against the stack-reversal reference.

Edge cases

- No parentheses → unchanged.
- Empty pair `()` → removed.
- Double nesting `((ab))` → the reversals cancel → `ab`.
- Siblings `(ab)(cd)` → each reversed independently → `badc`.

Bugs / debugging

None. It was verified against the $O(n^2)$ stack-of-strings reference on 1000 random balanced strings.

Alternatives considered

- Stack of strings (the $O(n^2)$ reference).
- A character stack that pops until `(`, reverses and pushes back. Also $O(n^2)$.

Complexity

- Time: $O(n)$.
- Space: $O(n)$.

Reusable pattern

Bracket matching via a stack, then "jump to the partner". Pre-computing partners turns nested structure into $O(1)$ jumps. A direction flag replaces physical reversal.

What to take away personally

If an operation is "reverse", ask whether you can instead *read* in the other direction. Here that removes all copying. This finishes a three-day bracket run (1807, 1190, 1614), all built on the same "match brackets / track depth" ideas.

Solution code (as submitted)

```
/**
 * 1190. Reverse Substrings Between Each Pair of Parentheses
 * https://leetcode.com/problems/reverse-substrings-between-each-pair-of-parentheses/
 *
 *  $O(n)$  "wormhole" walk. First match every bracket with its partner using a stack. Then walk
 * the string: on a letter, output it; on a bracket, jump to its partner and reverse the
 * walking direction. Entering a pair flips direction once (the inside is read reversed) and
 * leaving it flips back, which reproduces the effect of nested reversals without ever
 * reversing anything.
 *
 * @param {string} s
 * @return {string}
 */
var reverseParentheses = function (s) {
    const n = s.length;
    const partner = new Int32Array(n);
    const stack = [];
    for (let i = 0; i < n; i++) {
        if (s[i] === '(') stack.push(i);
        else if (s[i] === ')') {
            const j = stack.pop();
```

```
    partner[i] = j;
    partner[j] = i;
  }
}

const out = [];
for (let i = 0, dir = 1; i < n; i += dir) {
  if (s[i] === '(' || s[i] === ')') {
    i = partner[i];
    dir = -dir;
  } else {
    out.push(s[i]);
  }
}
return out.join('');
};
```

4.28 1614. Maximum Nesting Depth of the Parentheses

Field	Value
Daily Challenge date	2026-09-28
Solved	2026-09-28, inside the daily window (counted as a Daily Challenge)
Difficulty	Easy
Topics	String, Stack, Bracket Sequences
Link	https://leetcode.com/problems/maximum-nesting-depth-of-the-parentheses/
Result	Accepted, 168/168 tests, 0 ms, 53.6 MB (submission 2156484979)

What it asks (own words)

You are given an arithmetic-looking string whose parentheses are guaranteed to be balanced. Report how many parentheses deep the most deeply nested part of the string goes. A string with no parentheses has depth 0.

Key constraints

- Length 1..100, so any linear or even quadratic scan is fine.
- Input is guaranteed to be a valid parentheses string, so the depth counter can never go negative and no validation is needed.
- Other characters (digits, `+` `-` `*` `/`) are irrelevant noise.

Reasoning

A stack is the textbook tool for parentheses, but here nothing is ever read back out of the stack; only its *size* matters. The size of a stack of `(` at any moment is exactly "how many are open right now". So the stack collapses to a single integer counter, and the answer is the maximum value that counter ever reaches.

Algorithm

1. `depth = 0`, `best = 0`.
2. For each character:
 - `(` → `depth++`, then `best = max(best, depth)`.
 - `)` → `depth--`.
 - anything else → ignore.
3. Return `best`.

Why it works

At position i , `depth` equals the number of `(` before i that have not been closed yet, which is precisely the number of parentheses enclosing that position. Taking the maximum over all positions is the definition of the nesting depth. Updating `best` only after an increment is enough, because `depth` can only reach a new maximum right after an opening bracket.

JavaScript implementation details

- `for (const ch of s)` iterates characters directly; no index needed.
- Comparing `ch === '('` against a string literal is clearer than `charCodeAt` and fast enough at $n \leq 100$.

- `if (depth > best) best = depth` avoids a `Math.max` call per character (a micro-detail, not a requirement).

Edge cases

- No parentheses (`"1"`) $\rightarrow 0$.
- Sibling groups `()()()` $\rightarrow 1$, since siblings do not add depth.
- Maximum nesting at the limit: 50 `(` followed by 50 `)` $\rightarrow 50$.

Bugs / debugging

None on this problem. Examples passed locally and on LeetCode before submitting; accepted on the first submission.

Alternatives considered

- **Explicit stack:** push on `(`, pop on `)`, track the max stack size. Same $O(n)$ time but $O(n)$ space, and it stores data that is never used.
- **Prefix sum view:** map `(` $\rightarrow +1$, `)` $\rightarrow -1$, others $\rightarrow 0$; the answer is the maximum prefix sum. This is the same algorithm written in a different vocabulary and is a useful mental model for harder bracket problems.

Complexity

- Time: $O(n)$, one pass.
- Space: $O(1)$, two integers.

Reusable pattern

"Stack whose contents you never inspect $\rightarrow$ replace it with a counter." Many bracket problems only need the running balance (prefix sum of $+1/-1$). The same idea checks validity (the balance must never drop below 0 and must end at 0) and finds depth (max balance).

What to take away personally

Before reaching for a data structure, ask what information you actually read back from it. If only the size matters, a counter is enough.

Solution code (as submitted)

```
/**
 * 1614. Maximum Nesting Depth of the Parentheses
 * https://leetcode.com/problems/maximum-nesting-depth-of-the-parentheses/
 *
 * Single pass with a depth counter: '(' goes one level deeper, ')' comes back up.
 * The answer is the deepest level reached at any point.
 *
 * @param {string} s
 * @return {number}
 */
var maxDepth = function (s) {
  let depth = 0;
  let best = 0;
  for (const ch of s) {
    if (ch === '(') {
      depth++;
      if (depth > best) best = depth;
    }
  }
}
```

```
    } else if (ch == ')') {  
        depth--;  
    }  
}  
return best;  
};
```

4b. Patterns recurring across problems

The 28 problems cluster into a handful of reusable ideas. Each pattern below lists the problems where it appeared, the signal in the problem statement that should trigger it, and the core move.

1. Read the constraints first ("I / II" pairs)

Problems: 3875/3876, 3903/3904, 3870/3871, 3524/3525

Four pairs shared a statement and differed only in constraints or in one extra rule. The constraint picks the algorithm:

Pair	Part I	Part II	What changed
Uniform Parity	always true	compare <code>minOdd < minEven</code>	subtraction result must be ≥ 1
Smallest Stable Index	$O(n^2)$ scan ($n \leq 100$)	prefix max + suffix min, $O(n)$ ($n \leq 10^5$)	n
Count Commas	loop over $1..n$ ($n \leq 10^5$)	sum over thresholds, $O(\log n)$ ($n \leq 10^{15}$)	n
Find X Value	histogram per right end	segment tree with (prod, prefix histogram) nodes	point updates + suffix queries

Habit: the Part I solution is a free, trustworthy test oracle for Part II.

2. Prefix / suffix precomputation and "best so far"

Problems: 3904, 1477, 1614 (running balance)

Signal: every index needs an aggregate of everything to its left or right; or you need two non-overlapping pieces.

Move: one pass builds `prefix[i]` / `suffix[i]`, or a running best, then answer each index in $O(1)$.

3. Sliding window on positive arrays

Problems: 1477, 1658

Signal: contiguous subarray, target sum, **all values positive**. **Move:** grow the right end; shrink the left while the sum is too large. Each index enters and leaves once, $O(n)$. **Warning:** with zeros or negatives, switch to prefix sums with a hash map.

4. Complement / change of viewpoint

Problems: 1658 (remove ends $\Leftrightarrow$ keep middle), 3871 (commas per number $\Leftrightarrow$ numbers per comma), 835 (try shifts $\Leftrightarrow$ pairs vote for shifts), 3483 (enumerate choices $\Leftrightarrow$ enumerate answers), 1190 (reverse $\Leftrightarrow$ read backwards)

Signal: the direct search is exponential or quadratic, but there's an equivalent quantity that's easier to count.

Move: ask "what's left over?", "who contributes to what?", or "what is the answer space?".

5. Dynamic programming on sequences

Problems: 115, 940, 2472, 3414, 3524, 1621

Problem	State	Key trick
115 Distinct Subsequences	ways to build <code>t[0..j]</code>	iterate j backwards so each char of s is used once (0/1 knapsack style)

Problem	State	Key trick
940 Distinct Subsequences II	distinct subsequences ending in each letter	replace, don't add , to kill duplicates
2472 Palindrome Substrings	best count for prefix	only palindromes of length k or $k+1$ matter (shrink argument)
3414 Non-overlapping Intervals	(position, intervals left ≤ 4)	sort + binary search for the next compatible item; lexicographic tie-break carried in the DP value
3524 X Value I	residue histogram of subarrays ending here	tiny state ($k \leq 5$)
1621 Line Segments	ways with j segments in points $0..i$	prefix sums make transitions $O(1)$ (used as the test oracle)

6. Greedy on intervals, justified by structure

Problems: 1520, 3414 (contrast), 836

Signal: choose non-overlapping intervals to maximize count. **Move:** earliest-end / innermost-first greedy, **only after** proving the candidates have a nice structure (1520: nested-or-disjoint). When weights are involved (3414), greedy fails and DP is needed.

7. Bracket matching and stacks

Problems: 1614, 1807, 1190, 1096

Problem	What the "stack" became
1614 Nesting Depth	a single counter (only the size was needed)
1807 Bracket Pairs	no stack needed: brackets aren't nested, so just find the next <code>)</code>
1190 Reverse Substrings	a stack to pre-compute partners, then an $O(n)$ walk that flips direction
1096 Brace Expansion II	the call stack of a recursive-descent parser

8. Search with augmented state

Problem: 3568

Signal: "shortest number of moves" with extra conditions (items collected, fuel). **Move:** BFS over (position, bitmask of collected items, fuel). Prune with **dominance:** the same (position, mask) with less fuel is useless. **Hint:** a constraint like "at most 10 items" means a bitmask.

9. Trees: aggregate from children

Problem: 2265

Move: a post-order DFS returning a small tuple (sum, count). The parent combines children in $O(1)$.

10. Segment trees with custom merges

Problem: 3525

Move: define what a node summarizes, prove the merge is associative, and **respect order** if it isn't commutative (query left to right).

11. Math, geometry and counting

Problems: 3875, 3876 (parity algebra), 3871 (double counting), 1621 (bijection → binomial), 836 and 1401 (per-axis decomposition), 3550 and 3498 (digit / char arithmetic)

- Write the parity table: $E-O = O$, $O-O = E$, ...
- Axis-aligned geometry decomposes into independent 1-D checks: interval overlap `max(starts) < min(ends)`, and the closest point is a per-axis clamp.
- Compare **squared** distances to stay in exact integers.
- Insert gaps to turn " $\leq$ " into "<" (stars and bars), giving a single binomial coefficient.

12. JavaScript-specific lessons

Lesson	Where
Numbers are 64-bit floats: exact integers only up to $2^{53} \approx 9 \cdot 10^{15}$	3871 ($n \leq 10^{15}$ is safe), 115 (why huge intermediates don't break the answer), 3524 (counts $\approx 5 \cdot 10^9$ are safe)
Modular multiplication with a $\sim 10^9$ modulus overflows 2^{53} , so use <code>BigInt</code>	1621
<code>%</code> keeps the dividend's sign: <code>-3 % 2 === -1</code> ; add MOD before <code>%</code> after subtraction	940, 3875 test
<code>/</code> is float division, so use <code>Math.floor</code> for integer division	3483, 3550, 3568
<code>new Array(n).fill([])</code> shares one array, so use <code>.fill(null).map(() => [])</code>	3414
Typed arrays (<code>Int8Array</code> , <code>Int32Array</code>) for large flat state tables	3568, 835, 3525, 1190
<code>new Map(arrayOfPairs)</code> , <code>??</code> versus <code> </code>	1807
Build strings with an array + <code>join</code>	1807, 1190
<code>Intl.NumberFormat</code> as a real-world test oracle	3870
Default <code>.sort()</code> is lexicographic, so numbers need a comparator	1096, 3414

13. Testing lessons

- **Every non-trivial solution was checked against an independent brute force on random inputs** before submitting. This is why all 28 were accepted on the first submission.
- When a test fails, **check the test's assumption first** (3568: the stress test assumed a solvable grid).
- An oracle you can't read at a glance is not an oracle; replace it with the textbook version (1621).
- Tie-heavy random inputs (small weights) are what flush out tie-break bugs (3414).

5. Algorithms and data structures practiced

Technique	Problems
Dynamic programming (sequence, knapsack-style, counting)	115, 940, 2472, 3414, 3524, 1621
Sliding window	1477, 1658
Prefix / suffix aggregates	3903, 3904, 1477
Binary search (upper bound)	3414
Greedy with a structural proof	1520, 2472
BFS with bitmask state and dominance pruning	3568
Segment tree with a custom merge	3525
Tree DFS (post-order aggregates)	2265
Stacks, bracket matching, recursive-descent parsing	1614, 1190, 1807, 1096
Hash maps / sets / counting arrays	1807, 1096, 3483, 835
Combinatorics (bijection, binomials, Fermat inverse)	1621
Number theory / parity / modular arithmetic	3875, 3876, 3524, 3525, 940
Geometry (axis-aligned decomposition, clamping)	836, 1401
Math / counting (contribution technique)	3870, 3871
Simulation and digit/char arithmetic	3498, 3550, 3483

6. What I learned overall, and what to review

Overall lessons

- **Constraints choose the algorithm.** Four "I / II" pairs had near-identical statements; n or one extra rule changed everything.
- **Change the viewpoint before optimizing.** Several "hard-looking" problems collapsed once reframed: keep the middle instead of removing the ends (1658), count per threshold instead of per number (3871), let pairs vote for shifts (835), read backwards instead of reversing (1190).
- **Prove greedy choices.** 1520's greedy is safe only because its candidates are nested-or-disjoint, and 2472's shortcut is safe only because palindromes shrink.
- **JavaScript numbers are doubles.** Know the 2^{53} limit, use BigInt for modular multiplication, and watch % with negative numbers.
- **Test against a brute force.** Every solution was compared with an independent implementation on random inputs. That's why all 28 were accepted on the first submission, and it caught one wrong *test* assumption (3568).

Areas to review further (suggested order)

1. **Segment trees** (3525): rebuild one from scratch, then do a classic range-sum/point-update problem.
2. **Bitmask BFS / DP** (3568): the " $\leq 10-20$ items means bitmask" signal; try Shortest Path Visiting All Nodes (847).
3. **Weighted interval scheduling** (3414): first without the tie-break, then with it.
4. **Distinct-subsequence DPs** (115 vs 940): explain out loud why one *adds* and the other *replaces*.
5. **Combinatorial bijections** (1621): re-derive $C(n+k-1, 2k)$ on paper.

6. **Recursive-descent parsing** (1096): then Basic Calculator (224) with the same technique.
7. **Sliding window vs prefix-sum hash map** (1477, 1658): solve a variant with negative numbers.

7. Reusable techniques cheat sheet

Situation	Reach for
$n \leq \sim 100$	Brute force is fine; keep it as a test oracle
$n \leq 10^5$	$O(n)$ or $O(n \log n)$
$n \leq 10^{15}$	$O(\log n)$ or a formula: iterate thresholds/digits, not items
$\leq 10\text{--}20$ special items	Bitmask over subsets
Contiguous subarray, positive values, target sum	Sliding window
Contiguous subarray, arbitrary values	Prefix sums + hash map
"Two non-overlapping X"	Sweep with a best-X-so-far on the left
Aggregates to the left/right of every index	Prefix / suffix arrays
Choose non-overlapping weighted intervals	Sort by start, binary search next, skip/take DP
Count subsequences forming t	1-D DP, iterate j backwards
Count distinct subsequences	Per-last-character DP, replace instead of add
Brackets: depth / validity	A counter (running balance)
Brackets: nested reversal	Match partners with a stack, walk and flip direction
A grammar is given	Recursive descent, one function per precedence level
Point updates + range queries	Segment tree with an associative merge
Tree property "for every subtree"	Post-order DFS returning a tuple
Axis-aligned overlap	<code>max(starts) < min(ends)</code> on each axis
Closest point on a box	Clamp each coordinate
Modular product, mod $\approx 10^9$, in JS	BigInt
Modular subtraction	<code>(a - b + MOD) % MOD</code>

JavaScript snippets

```
const clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v));
const upperBound = (arr, x) => { let lo = 0, hi = arr.length;
  while (lo < hi) { const mid = (lo + hi) >> 1; if (arr[mid] > x) hi = mid; else lo = mid + 1; }
  return lo; };
// first index with arr[i] > x
const grid2d = (r, c, v) => Array.from({ length: r }, () => new Array(c).fill(v)); // no aliasing
const modPow = (b, e, m) => { let r = 1n; b %= m;
  for (; e > 0n; e >>= 1n) { if (e & 1n) r = r * b % m; b = b * b % m; } return r; };
const idx = (ch) => ch.charCodeAt(0) - 97; // 'a' -> 0
const intDiv = (a, b) => Math.floor(a / b); // '/' is float division in JS
```

8. LeetCode badge progress (verified on LeetCode)

Badge / milestone	Requirement	Start	Now	Status
September 2026 monthly badge	Complete every September daily (tickets can make up at most 3)	0/30	1/30	Not attainable: 27 days missed
Daily Challenge streak	Consecutive days completing the daily	0	1 day	In progress: needs a daily on Sept 29 and after
October 2026 monthly badge	Every October daily	—	—	Future activity required
50 Days / 100 Days badges (annual)	50 / 100 days with submissions in a year	0 days	1 active day	Future activity required
Earned during this project	—	—	—	None. The profile shows no badges.

Categorized status

- **Existing before the project:** none.
- **Earned during the project:** none. (Milestones: first accepted solutions, a 1-day streak, and the Sept 28 daily completed.)
- **In progress:** Daily streak (1 day).
- **Obtainable with future activity:** monthly badges from October onwards, and the 50/100-day badges.
- **Requires another real person:** none for LeetCode.

9. GitHub achievement progress (verified on GitHub)

Achievement	Requirement	Status
Quickdraw	Close an issue/PR within 5 minutes of opening	Existing before the project
Pull Shark	Merged pull requests	Not pursued: opening PRs only to earn a badge would be artificial
YOLO	Merge a PR without review	Not pursued, for the same reason
Starstruck	A repository receives 16 stars	Requires other real people to star the repo
Galaxy Brain	Accepted answers in GitHub Discussions	Requires other real people
Pair Extraordinaire	Co-authored commits merged in a PR	Not pursued
Public Sponsor	Sponsor someone	Requires spending money; not pursued
Earned during this project	—	None. The profile shows Quickdraw only.

10. GitHub repository summary

- **Repository:** `leetcode-daily-challenges` (public), <https://github.com/Rizwankhalid675/leetcode-daily-challenges>
- **Contents:** 28 problem folders (solution, tests, notes), a patterns guide, a results ledger with a generated status table, helper scripts, and this PDF.
- **Commit history** (meaningful units of work, no backdating; as of when this PDF was built):
 - `9881c3b` Add cross-problem patterns guide and final README
 - `e7be0f5` Solve Sept 22–27 problems (3525, 1658, 3550, 1096, 1807, 1190)
 - `06ad753` Solve Sept 15–21 problems (2472, 1621, 1477, 1520, 1401, 3498, 3524)
 - `976b648` Solve Sept 8–14 problems (3870, 3871, 2265, 3483, 3414, 835, 836)
 - `16bdaf5` Solve Sept 1–7 problems (3568, 3875, 3876, 3903, 3904, 115, 940)
 - `5bbedf1` Set up project and solve Sept 28 daily (1614)